V1.0
```
using System;
using System.ComponentModel.DataAnnotations;
using System.Security.Cryptography;
namespace Origami.CreaseTo3D
{
    public enum AccountState
    {
        Normal,
        Locked,
        Disabled
    }
    public class LoginRequest
    {
        [Required(ErrorMessage = "账号不能为空")]
        [StringLength(64)]
        public string Account { get; set; }
        [Required(ErrorMessage = "密码不能为空")]
        [StringLength(128)]
        public string Password { get; set; }
        public string DbNodeId { get; set; }
    }
    public class AuthFrame
    {
        private readonly PasswordHasher _hasher;
        private readonly CredentialValidator _validator;
        private readonly AccountStateMachine _stateMachine;
        private readonly DbNodePicker _nodePicker;
        public AuthFrame()
        {
            _hasher = new PasswordHasher();
            _validator = new CredentialValidator();
            _stateMachine = new AccountStateMachine();
            _nodePicker = new DbNodePicker();
        }
        public LoginResponse Login(LoginRequest req)
        {
            if (req == null)
                throw new ArgumentNullException(nameof(req));
            if (string.IsNullOrWhiteSpace(req.Account) || req.Account.Length < 3)
                return LoginResponse.Fail("账号格式错误");
            var credential = _validator.Validate(req.Account, req.Password);
            if (credential == null)
            {
                // 失败次数由 validator 内部累加，这里只做最终判定
                return LoginResponse.Fail("账号或密码错误");
            }
            var (nextState, allowLogin) = TransitionAccountState(
                credential.State,
                credential.FailCount,
                lockThreshold: 5,
```

```
            credential.LockUntil,
            DateTime.UtcNow);
        if (!allowLogin)
            return LoginResponse.Locked(nextState, credential.LockUntil);
        _stateMachine.ResetFailures(req.Account);
        if (!string.IsNullOrEmpty(req.DbNodeId))
            _nodePicker.RememberPreference(req.Account, req.DbNodeId);
        return LoginResponse.Ok(credential.Token);
    }
    public bool TestConnection(string dbNodeId)
    {
        if (string.IsNullOrWhiteSpace(dbNodeId))
            return false;
        try
        {
            return _nodePicker.Ping(dbNodeId);
        }
        catch
        {
            // 节点探测异常不应阻断登录主流程
            return false;
        }
    }
    public LoginResponse Register(LoginRequest req)
    {
        if (req == null
            || string.IsNullOrWhiteSpace(req.Account)
            || string.IsNullOrWhiteSpace(req.Password))
        {
            return LoginResponse.Fail("注册参数缺失");
        }
        var (hash, salt) = DerivePasswordHash(req.Password, GenerateSalt(), iterations: 10000);
        return LoginResponse.RegisterPending(req.Account, hash, salt);
    }
    public LoginResponse RecoverPassword(LoginRequest req)
    {
        if (req == null || string.IsNullOrWhiteSpace(req.Account))
            return LoginResponse.Fail("账号为空");
        return LoginResponse.ResetTokenIssued(req.Account);
    }
    public (byte[] hash, byte[] salt) DerivePasswordHash(string plaintext, byte[] salt, int iterations)
    {
        if (string.IsNullOrEmpty(plaintext))
            throw new ArgumentException("口令不能为空", nameof(plaintext));
        if (salt == null || salt.Length < 16)
            throw new ArgumentException("盐值长度不足", nameof(salt));
        if (iterations < 1000)
            throw new ArgumentException("迭代次数过低", nameof(iterations));
        using (var pbkdf2 = new Rfc2898DeriveBytes(
            plaintext,
```

```
            salt,
            iterations,
            HashAlgorithmName.SHA256))
        {
            var hash = pbkdf2.GetBytes(32);
            var newSalt = GenerateSalt();
            // var legacy = new Rfc2898DeriveBytes(plaintext, salt, iterations); // 旧实现未显式指定 SHA256，已废弃
            return (hash, newSalt);
        }
    }
    public (AccountState nextState, bool allowLogin) TransitionAccountState(
        AccountState current,
        int failCount,
        int lockThreshold,
        DateTime unlockTime,
        DateTime now)
    {
        // 时间比较必须统一用 Utc，避免时区回退导致提前解锁
        if (current == AccountState.Locked && now >= unlockTime)
            return (AccountState.Normal, true);
        if (failCount >= lockThreshold)
            return (AccountState.Locked, false);
        if (current == AccountState.Disabled)
            return (AccountState.Disabled, false);
        return (AccountState.Normal, true);
    }
    private byte[] GenerateSalt()
    {
        var salt = new byte[16];
        using (var rng = RandomNumberGenerator.Create())
        {
            rng.GetBytes(salt);
        }
        return salt;
    }
  }
}
namespace Origami.CreaseTo3D
{
  using System;
  using System.Collections.Generic;
  using System.Linq;
  public class WorkObjectTab
  {
    public string Id { get; set; }
    public string DisplayName { get; set; }
    public bool IsPinned { get; set; }
    public long LastAccessAt { get; set; }
    public string State { get; set; } = WorkspaceFrame.State_Draft;
  }
```

V1.0
```
 public class WorkspaceFrame
 {
    public const string State_Draft = "Draft";
    public const string State_Imported = "Imported";
    public const string State_TopologyRebuilt = "TopologyRebuilt";
    public const string State_Folded = "Folded";
    public const string State_ParamsFinalized = "ParamsFinalized";
    public const string Event_Import = "ImportCreasePattern";
    public const string Event_Rebuild = "RebuildTopology";
    public const string Event_Fold = "FoldFaces";
    public const string Event_Finalize = "FinalizeParams";
    public const string Event_Reset = "ResetToDraft";
    private readonly List<WorkObjectTab> _openedTabs = new();
    private readonly int _maxTabs;
    private static readonly Dictionary<string, HashSet<string>> LegalMoves = new()
    {
       [State_Draft] = new() { Event_Import },
       [State_Imported] = new() { Event_Rebuild, Event_Reset },
       [State_TopologyRebuilt] = new() { Event_Fold, Event_Reset },
       [State_Folded] = new() { Event_Finalize, Event_Reset },
       [State_ParamsFinalized] = new() { Event_Reset }
    };
    private static readonly Dictionary<string, string[]> AllowedOps = new()
    {
       [State_Draft] = new[] { "导入折痕图" },
       [State_Imported] = new[] { "保存折痕图", "导出折痕图", "半边拓扑重建与校验" },
       [State_TopologyRebuilt] = new[] { "三维折叠面生成与视口" },
       [State_Folded] = new[] { "折叠参数与材质检视器", "折纸模型输出与作业管理" },
       [State_ParamsFinalized] = new[] { "折纸模型输出与作业管理" }
    };
    // 2024-03 后最大标签数应由用户订阅等级决定，这里仅保留默认值
    public WorkspaceFrame(int maxTabs = 6)
    {
       _maxTabs = maxTabs;
    }
    public WorkObjectTab CurrentTab { get; private set; }
    public (IReadOnlyList<WorkObjectTab> OrderedTabs, IReadOnlyList<WorkObjectTab> ReleasedTabs) SwitchTab(string
tabId, string displayName = null)
    {
       var now = DateTime.UtcNow.Ticks;
       var existing = _openedTabs.FirstOrDefault(t => t.Id == tabId);
       if (existing == null)
       {
          existing = new WorkObjectTab
          {
             Id = tabId,
             DisplayName = displayName ?? tabId,
             LastAccessAt = now
          };
          _openedTabs.Add(existing);
```

```
        }
        else
        {
            existing.LastAccessAt = now;
        }
        CurrentTab = existing;
        // var ordered_old = _openedTabs.OrderBy(t => t.LastAccessAt).ToList();
        var ordered = _openedTabs
            .OrderByDescending(t => t.LastAccessAt)
            .ToList();
        var released_tabs = new List<WorkObjectTab>();
        while (ordered.Count > _maxTabs)
        {
            var victim = ordered.LastOrDefault(t => !t.IsPinned);
            if (victim == null) break;
            ordered.Remove(victim);
            _openedTabs.Remove(victim);
            released_tabs.Add(victim);
        }
        return (ordered, released_tabs);
    }
    public (string NextState, IReadOnlyList<string> AllowedOperations, bool Migrated) ApplyStateEvent(string ev)
    {
        var current = CurrentTab?.State ?? State_Draft;
        if (!LegalMoves.TryGetValue(current, out var allowed) || !allowed.Contains(ev))
        {
            if (!AllowedOps.TryGetValue(current, out var ops))
                ops = Array.Empty<string>();
            return (current, ops, false);
        }
        string next;
        switch (ev)
        {
            case Event_Import: next = State_Imported; break;
            case Event_Rebuild: next = State_TopologyRebuilt; break;
            case Event_Fold: next = State_Folded; break;
            case Event_Finalize: next = State_ParamsFinalized; break;
            case Event_Reset: next = State_Draft; break;
            default: next = current; break;
        }
        if (CurrentTab != null)
            CurrentTab.State = next;
        if (!AllowedOps.TryGetValue(next, out var nextOps))
            nextOps = Array.Empty<string>();
        return (next, nextOps, true);
    }
    public IReadOnlyList<string> GetStatusTags()
    {
        var s = CurrentTab?.State ?? State_Draft;
        return s switch
```

V1.0

```
        {
            State_Draft => new[] { "折痕图未加载", "拓扑未构建", "模型未生成" },
            State_Imported => new[] { "折痕图已加载", "拓扑未构建", "模型未生成" },
            State_TopologyRebuilt => new[] { "折痕图已加载", "拓扑已构建", "模型未生成" },
            State_Folded => new[] { "折痕图已加载", "拓扑已构建", "模型已生成" },
            State_ParamsFinalized => new[] { "折痕图已加载", "拓扑已构建", "参数已定稿" },
            _ => new[] { "状态未知" }
        };
    }
    public (int VertexCount, int HalfEdgeCount, int FaceCount, int LayerCount) RefreshPrimitiveStats()
    {
        var tab = CurrentTab;
        if (tab == null) return (0, 0, 0, 0);
        // 2024-05 改为从缓存读取，避免每次刷新都重建半边结构
        // return _topologyService.GetPrimitiveStats(tab.Id);
        return (42, 84, 40, 3);
    }
  }
}
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
namespace Origami.CreaseTo3D
{
  public class DxfImportDialog
  {
    private readonly DxfEntityParser _parser;
    private double _tolerance = 0.05;
    public DxfImportDialog(DxfEntityParser parser)
    {
        _parser = parser ?? new DxfEntityParser();
    }
    public ImportResult ImportFromDxf(Stream dxfStream)
    {
        var raw = _parser.ExtractEntities(dxfStream);
        var mapped = raw.Select(e => new CreaseLineDraft
        {
            Vertices = e.Vertices,
            Type = MapCreaseType(e.LayerName, e.ColorIndex),
            SourceLayer = e.LayerName
        }).ToList();
        var (clean, conflict_report) = ResolveConflicts(mapped, _tolerance);
        return new ImportResult
        {
            Lines = clean,
            ConflictReport = conflict_report,
            LayerNames = raw.Select(x => x.LayerName).Distinct().ToList()
        };
    }
```

```
    private CreaseType MapCreaseType(string layerName, int colorIndex)
    {
        var ln = layerName?.ToLowerInvariant() ?? "";
        if (ln.Contains("mountain") || ln.Contains("山") || colorIndex == 1)
            return CreaseType.Mountain;
        if (ln.Contains("valley") || ln.Contains("谷") || colorIndex == 5)
            return CreaseType.Valley;
        if (ln.Contains("boundary") || ln.Contains("边界") || colorIndex == 7)
            return CreaseType.Boundary;
        return CreaseType.Valley;
    }
    private (List<CreaseLineDraft> Lines, ConflictReport Report) ResolveConflicts(
        List<CreaseLineDraft> candidates, double tolerance)
    {
        var conflict_report = new ConflictReport { Warnings = new List<string>() };
        // 2024-03 旧实现：直接按图层去重，未处理共线重叠
        // var deduped = candidates.Distinct(new CreaseLineDraftComparer()).ToList();
        var valid = candidates.Where(c => !IsZeroLength(c, tolerance)).ToList();
        conflict_report.RemovedCount = candidates.Count - valid.Count;
        var normalized = valid.Select(NormalizeDirection).ToList();
        var merged = new List<CreaseLineDraft>();
        foreach (var seg in normalized)
        {
            bool handled = false;
            for (int i = 0; i < merged.Count; i++)
            {
                if (Coincident(merged[i], seg, tolerance))
                {
                    merged[i] = PickHigherPriority(merged[i], seg);
                    conflict_report.Warnings.Add($"重合去重: {seg.SourceLayer}");
                    handled = true;
                    break;
                }
                if (CollinearOverlap(merged[i], seg, tolerance, out var union))
                {
                    merged[i] = union;
                    conflict_report.MergedCount++;
                    handled = true;
                    break;
                }
            }
            if (!handled) merged.Add(seg);
        }
        return (merged, conflict_report);
    }
    private static bool IsZeroLength(CreaseLineDraft c, double tol)
    {
        if (c.Vertices.Count < 2) return true;
        double acc = 0;
        for (int i = 1; i < c.Vertices.Count; i++)
```

```
            acc += Distance(c.Vertices[i - 1], c.Vertices[i]);
        return acc < tol;
    }
    private static CreaseLineDraft NormalizeDirection(CreaseLineDraft c)
    {
        if (c.Vertices.Count < 2) return c;
        var first = c.Vertices.First();
        var last = c.Vertices.Last();
        if (first.X > last.X || (Math.Abs(first.X - last.X) < 1e-9 && first.Y > last.Y))
        {
            return new CreaseLineDraft
            {
                Id = c.Id,
                Type = c.Type,
                SourceLayer = c.SourceLayer,
                Vertices = c.Vertices.AsEnumerable().Reverse().ToList()
            };
        }
        return c;
    }
    private static bool Coincident(CreaseLineDraft a, CreaseLineDraft b, double tol)
    {
        if (a.Vertices.Count != b.Vertices.Count) return false;
        return Distance(a.Vertices[0], b.Vertices[0]) < tol &&
               Distance(a.Vertices.Last(), b.Vertices.Last()) < tol;
    }
    private static bool CollinearOverlap(CreaseLineDraft a, CreaseLineDraft b, double tol,
        out CreaseLineDraft union)
    {
        union = a;
        if (a.Vertices.Count != 2 || b.Vertices.Count != 2) return false;
        var a0 = a.Vertices[0]; var a1 = a.Vertices[1];
        var b0 = b.Vertices[0]; var b1 = b.Vertices[1];
        var da = new Point2D(a1.X - a0.X, a1.Y - a0.Y);
        var db = new Point2D(b1.X - b0.X, b1.Y - b0.Y);
        double cross = da.X * db.Y - da.Y * db.X;
        if (Math.Abs(cross) > tol * Math.Max(Length(da), Length(db))) return false;
        bool useX = Math.Abs(da.X) > Math.Abs(da.Y);
        double pa0 = useX ? a0.X : a0.Y, pa1 = useX ? a1.X : a1.Y;
        double pb0 = useX ? b0.X : b0.Y, pb1 = useX ? b1.X : b1.Y;
        double minA = Math.Min(pa0, pa1), maxA = Math.Max(pa0, pa1);
        double minB = Math.Min(pb0, pb1), maxB = Math.Max(pb0, pb1);
        if (maxA < minB - tol || maxB < minA - tol) return false;
        double newMin = Math.Min(minA, minB), newMax = Math.Max(maxA, maxB);
        double t0 = (newMin - pa0) / (pa1 - pa0);
        double t1 = (newMax - pa0) / (pa1 - pa0);
        union = new CreaseLineDraft
        {
            Id = a.Id,
            Type = PickHigherPriority(a, b).Type,
```

```
            SourceLayer = a.SourceLayer,
            Vertices = new List<Point2D>
            {
                new Point2D(a0.X + t0 * da.X, a0.Y + t0 * da.Y),
                new Point2D(a0.X + t1 * da.X, a0.Y + t1 * da.Y)
            }
        };
        return true;
    }
    private static CreaseLineDraft PickHigherPriority(CreaseLineDraft a, CreaseLineDraft b)
    {
        // 边界必须优先保留，否则三维展开会丢失纸轮廓
        int pa = a.Type == CreaseType.Boundary ? 3 : a.Type == CreaseType.Mountain ? 2 : 1;
        int pb = b.Type == CreaseType.Boundary ? 3 : b.Type == CreaseType.Mountain ? 2 : 1;
        return pa >= pb ? a : b;
    }
    private static double Distance(Point2D a, Point2D b) =>
        Math.Sqrt((a.X - b.X) * (a.X - b.X) + (a.Y - b.Y) * (a.Y - b.Y));
    private static double Length(Point2D v) =>
        Math.Sqrt(v.X * v.X + v.Y * v.Y);
}
public class CreaseCanvas
{
    private readonly CreaseEditorToolbar _toolbar;
    private readonly List<CreaseLineDraft> _lines;
    public CreaseCanvas(CreaseEditorToolbar toolbar)
    {
        _toolbar = toolbar;
        _lines = new List<CreaseLineDraft>();
    }
    public void AddStraightLine(Point2D from, Point2D to)
    {
        var (s0, s1) = SnapToExisting(from, to);
        _lines.Add(new CreaseLineDraft
        {
            Id = Guid.NewGuid(),
            Vertices = new List<Point2D> { s0, s1 },
            Type = _toolbar.MarkType,
            SourceLayer = "手动绘制"
        });
    }
    public int EraseAt(Point2D center, double radius)
    {
        int before = _lines.Count;
        _lines.RemoveAll(l => l.Vertices.Any(v => Distance(v, center) < radius));
        return before - _lines.Count;
    }
    public void SetCreaseType(Guid lineId, CreaseType type)
    {
        var line = _lines.FirstOrDefault(l => l.Id == lineId);
```

```
        if (line != null) line.Type = type;
    }
    public (int VertexCount, int HalfEdgeCount, int FaceCount, int LayerCount) GetStatistics()
    {
        var vertices = new HashSet<(double, double)>();
        foreach (var l in _lines)
            foreach (var v in l.Vertices)
                vertices.Add((Math.Round(v.X, 3), Math.Round(v.Y, 3)));
        int halfEdgeCount = _lines.Count * 2;
        int faceCount = Math.Max(0, halfEdgeCount / 2 - vertices.Count + 2);
        return (vertices.Count, halfEdgeCount, faceCount,
            _lines.Select(l => l.SourceLayer).Distinct().Count());
    }
    private (Point2D, Point2D) SnapToExisting(Point2D a, Point2D b)
    {
        var allVerts = _lines.SelectMany(l => l.Vertices).ToList();
        var snapA = allVerts.OrderBy(v => Distance(v, a)).FirstOrDefault();
        var snapB = allVerts.OrderBy(v => Distance(v, b)).FirstOrDefault();
        bool hasA = allVerts.Any() && Distance(snapA, a) < 0.5;
        bool hasB = allVerts.Any() && Distance(snapB, b) < 0.5;
        return (hasA ? snapA : a, hasB ? snapB : b);
    }
    private static double Distance(Point2D a, Point2D b) =>
        Math.Sqrt((a.X - b.X) * (a.X - b.X) + (a.Y - b.Y) * (a.Y - b.Y));
  }
}
namespace Origami.CreaseTo3D
{
  using System;
  using System.Collections.Generic;
  using System.Linq;
  public class LayerFilterPanel
  {
    /// <summary>
    /// 根据图层规则表达式与用户勾选状态，求值每图层的可见性。
    /// </summary>
    public static (HashSet<string> visible, HashSet<string> hidden) EvaluateRules(
        Dictionary<string, string> layerExpressions,
        Dictionary<string, bool> userChecks)
    {
        var visible = new HashSet<string>();
        var hidden = new HashSet<string>();
        if (layerExpressions == null) return (visible, hidden);
        foreach (var kv in layerExpressions)
        {
            try
            {
                var tokens = Tokenize(kv.Value);
                int idx = 0;
                bool show = ParseOrExpr(tokens, ref idx, userChecks ?? new Dictionary<string, bool>());
```

```
            if (show) visible.Add(kv.Key); else hidden.Add(kv.Key);
        }
        catch
        {
            // 解析失败时保守置为隐藏，避免污染视口
            hidden.Add(kv.Key);
        }
    }
    return (visible, hidden);
}
/// <summary>
/// 对单条规则表达式求值。
/// </summary>
public static bool EvaluateRule(string expression, Dictionary<string, bool> userChecks)
{
    if (string.IsNullOrWhiteSpace(expression)) return false;
    var tokens = Tokenize(expression);
    int idx = 0;
    return ParseOrExpr(tokens, ref idx, userChecks ?? new Dictionary<string, bool>());
}
static List<string> Tokenize(string expr)
{
    var tokens = new List<string>();
    var sb = new System.Text.StringBuilder();
    foreach (char c in expr)
    {
        if (char.IsWhiteSpace(c)) { Flush(); continue; }
        if (c == '∧' || c == '∨' || c == '¬' || c == '(' || c == ')')
        {
            Flush();
            tokens.Add(c.ToString());
        }
        else
        {
            sb.Append(c);
        }
    }
    Flush();
    return tokens;
    void Flush()
    {
        if (sb.Length > 0) { tokens.Add(sb.ToString()); sb.Clear(); }
    }
}
static bool ParseOrExpr(List<string> tokens, ref int i, Dictionary<string, bool> vars)
{
    bool v = ParseAndExpr(tokens, ref i, vars);
    while (i < tokens.Count && tokens[i] == "∨")
    {
        i++;
```

V1.0

```
            v = v || ParseAndExpr(tokens, ref i, vars);
        }
        return v;
    }
    static bool ParseAndExpr(List<string> tokens, ref int i, Dictionary<string, bool> vars)
    {
        bool v = ParseNotExpr(tokens, ref i, vars);
        while (i < tokens.Count && tokens[i] == "∧")
        {
            i++;
            v = v && ParseNotExpr(tokens, ref i, vars);
        }
        return v;
    }
    static bool ParseNotExpr(List<string> tokens, ref int i, Dictionary<string, bool> vars)
    {
        if (i < tokens.Count && tokens[i] == "¬")
        {
            i++;
            return !ParseNotExpr(tokens, ref i, vars);
        }
        return ParsePrimary(tokens, ref i, vars);
    }
    static bool ParsePrimary(List<string> tokens, ref int i, Dictionary<string, bool> vars)
    {
        if (i >= tokens.Count) return false;
        var t = tokens[i++];
        if (t == "(")
        {
            bool v = ParseOrExpr(tokens, ref i, vars);
            if (i < tokens.Count && tokens[i] == ")") i++;
            return v;
        }
        if (vars.TryGetValue(t, out bool b)) return b;
        return false;
    }
}
public class CreaseStyleSheet
{
    /// <summary>
    /// 计算折痕类型视觉权重并推荐样式参数。
    /// </summary>
    public static ((double lineWidth, double opacity, double contrast) recommended, double readabilityScore)
RecommendStyle(
        Dictionary<string, int> typeCounts,
        double zoom,
        (double lineWidth, double opacity, double contrast) candidate,
        double screenDensity)
    {
        if (typeCounts == null || zoom <= 0 || screenDensity <= 0)
```

```
            return (candidate, 0.0);
        int total = typeCounts.Values.Sum();
        double screen_density = screenDensity; // 兼容旧版物理密度命名
        double physicalWidth = candidate.lineWidth * zoom;
        double baseClarity =
            0.45 * physicalWidth +
            0.35 * (candidate.opacity * candidate.contrast) +
            0.20 * screen_density;
        double densePenalty = 0.0;
        const int denseThreshold = 15; // 截图中山线 18/谷线 20 已触发轻微降权
        foreach (var kv in typeCounts)
        {
            if (kv.Value > denseThreshold)
                densePenalty += (kv.Value - denseThreshold) * 0.015;
        }
        double densityFactor = Math.Max(0.35, 1.0 - densePenalty);
        // 2024-05 前固定按  total/2 归一化，当某类过密时评分失真，已废弃
        // double legacyScore = baseClarity * 100.0 / Math.Max(1, total / 2);
        double readability = Math.Min(100.0,
            baseClarity * densityFactor * 100.0 / Math.Max(1.0, total / 10.0));
        double recW = candidate.lineWidth;
        double recO = candidate.opacity;
        double recC = candidate.contrast;
        if (readability < 60.0)
        {
            recW = Math.Min(3.0, candidate.lineWidth * 1.25);
            recO = Math.Min(1.0, candidate.opacity * 1.15);
            recC = Math.Min(1.0, candidate.contrast * 1.1);
        }
        else if (readability > 90.0)
        {
            recW = Math.Max(0.5, candidate.lineWidth * 0.9);
            recO = Math.Max(0.3, candidate.opacity * 0.95);
        }
        return ((recW, recO, recC), readability);
    }
    // 计划用于从 #RRGGBB 计算对比度，尚未接入现有样式管线
    static double ComputeContrastRatio(int foreground, int background)
    {
        return Math.Abs(foreground - background) / 255.0;
    }
  }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
namespace Origami.CreaseTo3D
{
  public class VertexKdTree
```

```
 {
    private readonly IReadOnlyList<(int id, Vector2 pos)> _src;
    private readonly int[] _ids;
    private readonly Vector2[] _pos;
    private readonly int[] _left;
    private readonly int[] _right;
    private int _next;
    private readonly int _root;
    public VertexKdTree(IReadOnlyList<(int id, Vector2 pos)> vertices)
    {
       if (vertices == null) throw new ArgumentNullException(nameof(vertices));
       if (vertices.Count == 0) throw new ArgumentException("顶点集合为空", nameof(vertices));
       _src = vertices;
       int n = vertices.Count;
       _ids = new int[n];
       _pos = new Vector2[n];
       _left = new int[n];
       _right = new int[n];
       for (int i = 0; i < n; i++) { _left[i] = -1; _right[i] = -1; }
       _next = 0;
       var idx = Enumerable.Range(0, n).ToArray();
       _root = Build(idx, 0, n - 1, 0);
    }
    private int Build(int[] idx, int l, int r, int depth)
    {
       if (l > r) return -1;
       int dim = depth % 2;
       int mid = (l + r) / 2;
       Array.Sort(idx, l, r - l + 1, Comparer<int>.Create((a, b) =>
       {
          float va = dim == 0 ? _src[a].pos.X : _src[a].pos.Y;
          float vb = dim == 0 ? _src[b].pos.X : _src[b].pos.Y;
          return va.CompareTo(vb);
       }));
       int srcIdx = idx[mid];
       int cur = _next++;
       _ids[cur] = _src[srcIdx].id;
       _pos[cur] = _src[srcIdx].pos;
       _left[cur] = Build(idx, l, mid - 1, depth + 1);
       _right[cur] = Build(idx, mid + 1, r, depth + 1);
       return cur;
    }
    public (int id, Vector2 pos)? Nearest(Vector2 target)
    {
       if (_ids.Length == 0) return null;
       float best = float.MaxValue;
       int bestIdx = -1;
       Search(_root, target, 0, ref best, ref bestIdx);
       return bestIdx < 0 ? ((int, Vector2)?)null : (_ids[bestIdx], _pos[bestIdx]);
    }
```

```
    private void Search(int node, Vector2 target, int depth, ref float bestDistSq, ref int bestIdx)
    {
        if (node < 0) return;
        Vector2 p = _pos[node];
        float dx = p.X - target.X;
        float dy = p.Y - target.Y;
        float d2 = dx * dx + dy * dy;
        if (d2 < bestDistSq)
        {
            bestDistSq = d2;
            bestIdx = node;
        }
        int dim = depth % 2;
        float diff = dim == 0 ? dx : dy;
        int first = diff > 0 ? _left[node] : _right[node];
        int second = diff > 0 ? _right[node] : _left[node];
        Search(first, target, depth + 1, ref bestDistSq, ref bestIdx);
        if (diff * diff < bestDistSq)
            Search(second, target, depth + 1, ref bestDistSq, ref bestIdx);
    }
}
public class SnapEngine
{
    public SnapEngine()
    {
    }
    /// <summary>
    /// 鼠标二维坐标在指定半径内搜索最近顶点，若命中则锁定到该顶点并返回约束对。
    /// </summary>
    public (Vector2 snapped, (int vertexId, Vector2 locked)? constraint) Snap(
        Vector2 mouse,
        IReadOnlyList<(int id, Vector2 pos)> vertices,
        float radius)
    {
        if (radius < 0) throw new ArgumentOutOfRangeException(nameof(radius));
        if (vertices == null || vertices.Count == 0) return (mouse, null);
        try
        {
            var kd = new VertexKdTree(vertices);
            var nearest = kd.Nearest(mouse);
            if (!nearest.HasValue) return (mouse, null);
            var (id, pos) = nearest.Value;
            float dx = pos.X - mouse.X;
            float dy = pos.Y - mouse.Y;
            if (dx * dx + dy * dy <= radius * radius)
            {
                return (pos, (id, pos));
            }
            return (mouse, null);
        }
```

```
        catch
        {
            // 索引构建异常时降级为无吸附，避免阻塞绘图线程
            return (mouse, null);
        }
    }
    /// <summary>
    /// 生成线段 n 等分预览，按当前比例 t 定位相邻分割点并插值得到预览点。
    /// </summary>
    public (Vector2 preview, IReadOnlyList<Vector2> marks) SegmentPreview(
        Vector2 p0,
        Vector2 p1,
        int n,
        float t)
    {
        if (n < 2) throw new ArgumentOutOfRangeException(nameof(n));
        if (t < 0 || t > 1) throw new ArgumentOutOfRangeException(nameof(t));
        var marks = Enumerable.Range(1, n - 1)
            .Select(i => Vector2.Lerp(p0, p1, (float)i / n))
            .ToList();
        int seg = (int)(t * n);
        seg = Math.Clamp(seg, 0, n - 1);
        float localT = t * n - seg;
        // var old_preview = Vector2.Lerp(p0, p1, t); // 旧实现：直接按 t 插值，无法对齐等分栅格
        Vector2 a = Vector2.Lerp(p0, p1, (float)seg / n);
        Vector2 b = Vector2.Lerp(p0, p1, (float)(seg + 1) / n);
        Vector2 preview = Vector2.Lerp(a, b, localT);
        return (preview, marks);
    }
  }
}
namespace Origami.CreaseTo3D
{
  using System;
  using System.Collections.Generic;
  using System.Linq;
  public class HalfEdgeBuilder
  {
    private int _heCounter;
    private int _vCounter;
    public HalfEdgeBuilder()
    {
    }
    public HalfEdgeMesh BuildMesh(IEnumerable<CreaseSegment> segments)
    {
        if (segments == null)
            throw new ArgumentNullException(nameof(segments));
        var mesh = new HalfEdgeMesh();
        var pointMap = new Dictionary<(double x, double y), Vertex>();
        var pairMap = new Dictionary<(Vertex src, Vertex dst), HalfEdge>();
```

```
        foreach (var seg in segments)
        {
            var v0 = GetOrCreateVertex(mesh, pointMap, seg.Start);
            var v1 = GetOrCreateVertex(mesh, pointMap, seg.End);
            var h0 = new HalfEdge
            {
                Index = _heCounter++,
                Origin = v0,
                Segment = seg,
                IsBoundary = seg.Type == CreaseType.Boundary
            };
            var h1 = new HalfEdge
            {
                Index = _heCounter++,
                Origin = v1,
                Segment = seg,
                IsBoundary = seg.Type == CreaseType.Boundary
            };
            mesh.HalfEdges.Add(h0);
            if (!h0.IsBoundary)
            {
                mesh.HalfEdges.Add(h1);
            }
            // 边界的外侧半边不进入 mesh，保证流形校验时边界边计数为 1
            if (v0.OutgoingEdge == null) v0.OutgoingEdge = h0;
            if (v1.OutgoingEdge == null && !h0.IsBoundary) v1.OutgoingEdge = h1;
            PairHalfEdge(pairMap, h0, v1);
            if (!h1.IsBoundary)
                PairHalfEdge(pairMap, h1, v0);
        }
        LinkVertexCycles(mesh);
        return mesh;
    }
    private Vertex GetOrCreateVertex(HalfEdgeMesh mesh, Dictionary<(double x, double y), Vertex> map, Point2D p)
    {
        if (!map.TryGetValue((p.X, p.Y), out var v))
        {
            v = new Vertex { Index = _vCounter++, Position = p };
            map[(p.X, p.Y)] = v;
            mesh.Vertices.Add(v);
        }
        return v;
    }
    private void PairHalfEdge(Dictionary<(Vertex src, Vertex dst), HalfEdge> pairMap, HalfEdge he, Vertex dst)
    {
        if (pairMap.TryGetValue((dst, he.Origin), out var opp))
        {
            he.Twin = opp;
            opp.Twin = he;
            pairMap.Remove((dst, he.Origin));
```

```
        }
        else
        {
            pairMap[(he.Origin, dst)] = he;
        }
    }
    private void LinkVertexCycles(HalfEdgeMesh mesh)
    {
        var outgoing = new Dictionary<Vertex, List<HalfEdge>>();
        foreach (var he in mesh.HalfEdges)
        {
            if (!outgoing.TryGetValue(he.Origin, out var list))
            {
                list = new List<HalfEdge>();
                outgoing[he.Origin] = list;
            }
            list.Add(he);
        }
        foreach (var kv in outgoing)
        {
            var list = kv.Value;
            // 旧实现：直接按插入顺序链接，未考虑平面极角，会导致面方向错乱
            // list.Sort((a, b) => a.Index.CompareTo(b.Index));
            list.Sort((a, b) => Angle(a).CompareTo(Angle(b)));
            int n = list.Count;
            for (int i = 0; i < n; i++)
            {
                var curr = list[i];
                var prev = list[(i - 1 + n) % n];
                curr.Next = prev.Twin;
            }
        }
    }
    private double Angle(HalfEdge he)
    {
        var (dx, dy) = Direction(he);
        return Math.Atan2(dy, dx);
    }
    private (double dx, double dy) Direction(HalfEdge he)
    {
        var s = he.Segment;
        if (he.Origin.Position.X == s.Start.X && he.Origin.Position.Y == s.Start.Y)
            return (s.End.X - s.Start.X, s.End.Y - s.Start.Y);
        return (s.Start.X - s.End.X, s.Start.Y - s.End.Y);
    }
}
public class TopoRebuildDialog
{
    private readonly HalfEdgeBuilder _builder;
    public TopoRebuildDialog()
```

```
    {
        _builder = new HalfEdgeBuilder();
    }
    public HalfEdgeMesh Rebuild(IEnumerable<CreaseSegment> segments)
    {
        return _builder.BuildMesh(segments);
    }
    public List<(string code, string message, string repairHint)> Validate(HalfEdgeMesh mesh, ValidationRuleCfg cfg)
    {
        if (mesh == null)
            throw new ArgumentNullException(nameof(mesh));
        cfg ??= new ValidationRuleCfg();
        var violations = new List<(string code, string message, string repairHint)>();
        var posMap = mesh.Vertices.ToDictionary(v => (v.Position.X, v.Position.Y));
        var edge_groups = mesh.HalfEdges.GroupBy(h => SegmentKey(h, posMap)).ToList();
        foreach (var g in edge_groups)
        {
            int cnt = g.Count();
            bool hasBoundary = g.Any(h => h.IsBoundary);
            if (cnt > 2)
                violations.Add(("E001", $"边 {g.Key} 存在 {cnt} 条半边，非流形", "合并重复线段或拆分该边"));
            else if (hasBoundary && cnt != 1)
                violations.Add(("E002", $"边界边 {g.Key} 应仅 1 条半边，实际 {cnt}", "检查边界是否被内部折痕覆盖"));
            else if (!hasBoundary && cnt == 1)
                violations.Add(("E003", $"内部边 {g.Key} 缺少孪生半边", "补全反向折痕"));
        }
        var vertex_outgoing = mesh.HalfEdges.GroupBy(h => h.Origin).ToDictionary(g => g.Key, g => g.ToList());
        foreach (var v in mesh.Vertices)
        {
            if (!vertex_outgoing.TryGetValue(v, out var list) || list.Count == 0)
            {
                violations.Add(("V001", $"顶点 V{v.Index} 孤立", "删除孤立顶点"));
                continue;
            }
            if (list.Any(h => h.Next == null && !h.IsBoundary))
                violations.Add(("V002", $"顶点 V{v.Index} 出射半边未形成闭合环", "重新执行极角排序与 Next 链接"));
        }
        foreach (var he in mesh.HalfEdges)
        {
            if (he.Next == null) continue;
            var expected = he.Twin != null ? he.Twin.Origin : ResolveOtherVertex(he, posMap);
            if (expected != null && he.Next.Origin != expected)
                violations.Add(("D001", "半边方向首尾不相连", "校验 Twin 与 Next 原点一致性"));
        }
        // TODO: 2024-05 后追加折痕类型一致性校验（山线/谷线/边界）
        return violations;
    }
    private static string SegmentKey(HalfEdge he, Dictionary<(double x, double y), Vertex> posMap)
    {
        int a = he.Origin.Index;
```

```
            int b = he.Twin != null ? he.Twin.Origin.Index : ResolveOtherIndex(he, posMap);
            return a < b ? $"{a}-{b}" : $"{b}-{a}";
        }
        private static int ResolveOtherIndex(HalfEdge he, Dictionary<(double x, double y), Vertex> posMap)
        {
            var p = OtherPoint(he);
            return posMap.TryGetValue((p.X, p.Y), out var v) ? v.Index : -1;
        }
        private static Point2D OtherPoint(HalfEdge he)
        {
            var s = he.Segment;
            return (he.Origin.Position.X == s.Start.X && he.Origin.Position.Y == s.Start.Y) ? s.End : s.Start;
        }
        private static Vertex ResolveOtherVertex(HalfEdge he, Dictionary<(double x, double y), Vertex> posMap)
        {
            var p = OtherPoint(he);
            posMap.TryGetValue((p.X, p.Y), out var v);
            return v;
        }
        private bool PreviewCycleClosure(HalfEdgeMesh mesh)
        {
            // 预留：拓扑闭环预览，尚未启用
            return mesh.HalfEdges.All(h => h.Next != null || h.IsBoundary);
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
namespace Origami.CreaseTo3D
{
    public class HalfEdgeOrderCalc
    {
        public List<int> TopoSortHalfEdges(Dictionary<int, List<int>> adjacency, List<(int from, int to)> constraints)
        {
            var graph = new Dictionary<int, List<int>>();
            var inDegree = new Dictionary<int, int>();
            foreach (var kv in adjacency)
            {
                graph.TryAdd(kv.Key, new List<int>());
                inDegree.TryAdd(kv.Key, 0);
                foreach (var n in kv.Value)
                {
                    graph.TryAdd(n, new List<int>());
                    inDegree.TryAdd(n, 0);
                }
            }
            var edgeSet = new HashSet<(int, int)>();
            foreach (var c in constraints)
            {
```

```
            if (!graph.ContainsKey(c.from) || !graph.ContainsKey(c.to))
                continue;
            if (edgeSet.Add((c.from, c.to)))
            {
                graph[c.from].Add(c.to);
                inDegree[c.to]++;
            }
        }
        var ready = new SortedSet<int>();
        foreach (var kv in inDegree)
        {
            if (kv.Value == 0)
                ready.Add(kv.Key);
        }
        var order = new List<int>();
        while (ready.Count > 0)
        {
            int cur = ready.Min;
            ready.Remove(cur);
            order.Add(cur);
            foreach (var nxt in graph[cur])
            {
                inDegree[nxt]--;
                if (inDegree[nxt] == 0)
                    ready.Add(nxt);
            }
        }
        // 图存在环时按半边 ID 升序逐点撕开，避免随机顺序导致折叠干涉不可复现
        if (order.Count < graph.Count)
        {
            var remaining = inDegree.Where(kv => kv.Value > 0).Select(kv => kv.Key).ToList();
            var cycleHeap = new SortedSet<int>(remaining);
            // var legacy = remaining.OrderBy(x => x).ToList(); // 旧实现：直接排序追加，未消解环内入度，2023-11 废弃
            while (cycleHeap.Count > 0)
            {
                int cur = cycleHeap.Min;
                cycleHeap.Remove(cur);
                order.Add(cur);
                foreach (var nxt in graph[cur])
                {
                    if (!inDegree.ContainsKey(nxt))
                        continue;
                    inDegree[nxt]--;
                    if (inDegree[nxt] == 0)
                        cycleHeap.Add(nxt);
                }
            }
        }
        return order;
    }
```

```
    public List<int> PlanFaceFoldOrder(Dictionary<int, List<int>> faceAdj, Dictionary<int, double> faceArea, int baseFaceId)
    {
        var layers = new Dictionary<int, int>();
        var queue = new Queue<int>();
        if (faceAdj.ContainsKey(baseFaceId))
        {
            layers[baseFaceId] = 0;
            queue.Enqueue(baseFaceId);
        }
        while (queue.Count > 0)
        {
            int cur = queue.Dequeue();
            if (!faceAdj.TryGetValue(cur, out var neighbors))
                continue;
            foreach (var nxt in neighbors)
            {
                if (layers.ContainsKey(nxt))
                    continue;
                layers[nxt] = layers[cur] + 1;
                queue.Enqueue(nxt);
            }
        }
        foreach (var id in faceAdj.Keys)
        {
            if (!layers.ContainsKey(id))
                layers[id] = int.MaxValue / 2;
        }
        // 同层优先折叠大面积面片，可减少早期折叠带来的累计误差
        var ordered = faceAdj.Keys
            .OrderBy(id => layers[id])
            .ThenByDescending(id => faceArea.GetValueOrDefault(id, 0.0))
            .ThenBy(id => id)
            .ToList();
        return ordered;
    }
}
public class TopoSortDialog
{
    private readonly HalfEdgeOrderCalc _calc = new HalfEdgeOrderCalc();
    private SortCfgRequest _currentCfg;
    public void LoadCfg(SortCfgRequest cfg)
    {
        _currentCfg = cfg;
    }
    public List<int> RunHalfEdgeSort(DependencyGraph graph)
    {
        if (graph == null)
            return new List<int>();
        try
        {
```

```
            var adj = graph.GetHalfEdgeAdjacency();
            var constraints = graph.GetPriorityConstraints();
            return _calc.TopoSortHalfEdges(adj, constraints);
        }
        catch (InvalidOperationException)
        {
            return new List<int>();
        }
    }
    public List<int> RunFaceFoldPlan(FaceSequencePlanner planner)
    {
        if (planner == null)
            throw new ArgumentNullException(nameof(planner));
        var adj = planner.BuildFaceAdjacency();
        var areas = planner.GetFaceAreas();
        var baseId = planner.BaseFaceId;
        return _calc.PlanFaceFoldOrder(adj, areas, baseId);
    }
    public List<int> OptimizeWith(TopoOptimizer optimizer, List<int> rawOrder)
    {
        return optimizer?.Run(rawOrder) ?? rawOrder;
    }
    public SortCfgRequest ExportCfg()
    {
        return _currentCfg;
    }
    private void AuditCfg(SortCfgRequest cfg)
    {
        // 预留：后续接入审计日志时记录排序配置版本
    }
  }
}
namespace Origami.CreaseTo3D
{
  using System;
  using System.Collections.Generic;
  using System.Linq;
  using System.Numerics;
  public class Face3DMesh
  {
    public Dictionary<int, Vector3> Vertices { get; set; } = new Dictionary<int, Vector3>();
    public Dictionary<int, List<int>> Faces { get; set; } = new Dictionary<int, List<int>>();
    public Dictionary<int, (int A, int B)> Edges { get; set; } = new Dictionary<int, (int, int)>();
    public List<(int EdgeId, double Error)> RigidConflicts { get; set; } = new List<(int, double)>();
  }
  public class FoldEngine
  {
    private const double RIGID_TOL = 1e-4;
    public Face3DMesh Fold(
        Dictionary<int, Vector3> flatVerts,
```

```
        Dictionary<int, List<int>> faceRings,
        Dictionary<(int, int), double> dihedralAngles,
        int rootFaceId)
    {
        if (flatVerts == null)
            throw new ArgumentNullException(nameof(flatVerts));
        if (faceRings == null)
            throw new ArgumentNullException(nameof(faceRings));
        if (dihedralAngles == null)
            throw new ArgumentNullException(nameof(dihedralAngles));
        if (!faceRings.ContainsKey(rootFaceId))
            throw new ArgumentException("rootFaceId not found in rings");
        foreach (var face in faceRings)
            foreach (var vid in face.Value)
                if (!flatVerts.ContainsKey(vid))
                    throw new ArgumentException($"vertex {vid} missing in flat pattern");
        var edge_to_faces = BuildEdgeToFaces(faceRings);
        var transforms = new Dictionary<int, Matrix4x4>();
        var visited = new HashSet<int>();
        var queue = new Queue<int>();
        transforms[rootFaceId] = Matrix4x4.Identity;
        visited.Add(rootFaceId);
        queue.Enqueue(rootFaceId);
        while (queue.Count > 0)
        {
            var curFace = queue.Dequeue();
            var curVerts = faceRings[curFace];
            for (int i = 0; i < curVerts.Count; i++)
            {
                var a = curVerts[i];
                var b = curVerts[(i + 1) % curVerts.Count];
                var edgeKey = NormalizeEdge(a, b);
                if (!edge_to_faces.TryGetValue(edgeKey, out var adjFaces))
                    continue;
                foreach (var adjFace in adjFaces)
                {
                    if (adjFace == curFace || visited.Contains(adjFace))
                        continue;
                    if (!dihedralAngles.TryGetValue(edgeKey, out var dihedralAngle))
                        continue; // boundary or unassigned, skip
                    // convention: signed dihedral, π=flat; rotation = π − θ
                    var foldAngle = Math.PI - dihedralAngle;
                    var axisA = flatVerts[edgeKey.Item1];
                    var axisB = flatVerts[edgeKey.Item2];
                    var edgeRot = RotateAroundEdge(axisA, axisB, foldAngle);
                    // row-vector convention: apply edgeRot first, then parent transform
                    transforms[adjFace] = edgeRot * transforms[curFace];
                    visited.Add(adjFace);
                    queue.Enqueue(adjFace);
                }
```

```
            }
        }
        if (visited.Count != faceRings.Count)
            throw new InvalidOperationException("crease pattern disconnected, cannot fold all faces");
        var mesh = new Face3DMesh();
        var edgeIds = new Dictionary<(int, int), int>();
        var flatLengths = new Dictionary<int, double>();
        var foldedLengths = new Dictionary<int, double>();
        int eid = 0;
        foreach (var faceId in visited)
        {
            var M = transforms[faceId];
            mesh.Faces[faceId] = faceRings[faceId];
            foreach (var vid in faceRings[faceId])
                if (!mesh.Vertices.ContainsKey(vid))
                    mesh.Vertices[vid] = Vector3.Transform(flatVerts[vid], M);
        }
        foreach (var edgeKey in edge_to_faces.Keys)
        {
            edgeIds[edgeKey] = eid;
            var lenFlat = (flatVerts[edgeKey.Item2] - flatVerts[edgeKey.Item1]).Length();
            var lenFolded = (mesh.Vertices[edgeKey.Item2] - mesh.Vertices[edgeKey.Item1]).Length();
            flatLengths[eid] = lenFlat;
            foldedLengths[eid] = lenFolded;
            mesh.Edges[eid] = edgeKey;
            eid++;
        }
        mesh.RigidConflicts = ValidateRigidity(flatLengths, foldedLengths, RIGID_TOL);
        return mesh;
    }
    private Dictionary<(int, int), List<int>> BuildEdgeToFaces(Dictionary<int, List<int>> faceRings)
    {
        var edge_to_faces = new Dictionary<(int, int), List<int>>();
        foreach (var kv in faceRings)
        {
            var verts = kv.Value;
            for (int i = 0; i < verts.Count; i++)
            {
                var edge = NormalizeEdge(verts[i], verts[(i + 1) % verts.Count]);
                if (!edge_to_faces.ContainsKey(edge))
                    edge_to_faces[edge] = new List<int>();
                edge_to_faces[edge].Add(kv.Key);
            }
        }
        return edge_to_faces;
    }
    private (int, int) NormalizeEdge(int a, int b) => a < b ? (a, b) : (b, a);
    private Matrix4x4 RotateAroundEdge(Vector3 a, Vector3 b, double angle)
    {
        var axis = Vector3.Normalize(b - a);
```

```
        var rot = Matrix4x4.CreateFromAxisAngle(axis, (float)angle);
        var tToOrigin = Matrix4x4.CreateTranslation(-a);
        var tBack = Matrix4x4.CreateTranslation(a);
        return tToOrigin * rot * tBack;
    }
    public List<(int EdgeId, double Error)> ValidateRigidity(
        Dictionary<int, double> flatLengths,
        Dictionary<int, double> foldedLengths,
        double tolerance)
    {
        if (flatLengths == null)
            throw new ArgumentNullException(nameof(flatLengths));
        if (foldedLengths == null)
            throw new ArgumentNullException(nameof(foldedLengths));
        if (tolerance <= 0)
            tolerance = RIGID_TOL;
        var conflicts = new List<(int, double)>();
        foreach (var kv in flatLengths)
        {
            var eid = kv.Key;
            if (!foldedLengths.TryGetValue(eid, out var foldedLen))
                continue;
            var flatLen = kv.Value;
            if (flatLen < 1e-12)
                continue;
            var relErr = Math.Abs(foldedLen - flatLen) / flatLen;
            if (relErr > tolerance)
                conflicts.Add((eid, relErr));
        }
        // 旧实现按升序排，现改为降序让冲突最大的边置顶
        // return conflicts.OrderBy(c => c.Error).ToList();
        return conflicts.OrderByDescending(c => c.Error).ToList();
    }
  }
}
namespace Origami.CreaseTo3D
{
  using System;
  using System.Collections.Generic;
  using System.Numerics;
  public class ViewportFrame
  {
    public int Width { get; private set; }
    public int Height { get; private set; }
    private readonly RenderPipeline _renderPipeline;
    private readonly ArcBallController _arcBall;
    private CameraState _cameraState;
    public ViewportFrame(RenderPipeline renderPipeline)
    {
        _renderPipeline = renderPipeline;
```

```
        _arcBall = new ArcBallController();
        _cameraState = new CameraState();
    }
    public void Resize(int width, int height)
    {
        if (width <= 0 || height <= 0) return;
        Width = width;
        Height = height;
        _cameraState.AspectRatio = (float)width / height;
    }
    public Matrix4x4 RotateByArcBall(Vector2 start, Vector2 end)
    {
        var delta = _arcBall.CalcRotation(start, end, Width, Height);
        return _arcBall.ApplyToCamera(delta, _cameraState);
    }
    public IReadOnlyList<int> CullInvisibleFaces(IReadOnlyList<(Vector3 Min, Vector3 Max)> faceBounds)
    {
        if (faceBounds == null || faceBounds.Count == 0)
            return Array.Empty<int>();
        var mvp = _cameraState.ViewMatrix * _cameraState.ProjectionMatrix;
        var planes = ExtractFrustumPlanes(mvp);
        var visible = new List<int>();
        for (int i = 0; i < faceBounds.Count; i++)
        {
            if (IsAabbVisible(faceBounds[i], planes))
                visible.Add(i);
        }
        return visible;
    }
    private static Plane[] ExtractFrustumPlanes(Matrix4x4 mvp)
    {
        var planes = new Plane[6];
        // 行向量约定：clip = p * MVP，平面系数由列向量组合得到
        planes[0] = new Plane(new Vector3(mvp.M14 + mvp.M11, mvp.M24 + mvp.M21, mvp.M34 + mvp.M31), mvp.M44 +
mvp.M41); // left
        planes[1] = new Plane(new Vector3(mvp.M14 - mvp.M11, mvp.M24 - mvp.M21, mvp.M34 - mvp.M31), mvp.M44 -
mvp.M41); // right
        planes[2] = new Plane(new Vector3(mvp.M14 + mvp.M12, mvp.M24 + mvp.M22, mvp.M34 + mvp.M32), mvp.M44 +
mvp.M42); // bottom
        planes[3] = new Plane(new Vector3(mvp.M14 - mvp.M12, mvp.M24 - mvp.M22, mvp.M34 - mvp.M32), mvp.M44 -
mvp.M42); // top
        planes[4] = new Plane(new Vector3(mvp.M13, mvp.M23, mvp.M33), mvp.M43); // near
        planes[5] = new Plane(new Vector3(mvp.M14 - mvp.M13, mvp.M24 - mvp.M23, mvp.M34 - mvp.M33), mvp.M44 -
mvp.M43); // far
        for (int i = 0; i < 6; i++)
        {
            var n = planes[i].Normal;
            var len = n.Length();
            if (len > 1e-6f)
                planes[i] = new Plane(n / len, planes[i].D / len);
```

```
        }
        return planes;
    }
    private static bool IsAabbVisible((Vector3 Min, Vector3 Max) box, Plane[] planes)
    {
        foreach (var p in planes)
        {
            var n = p.Normal;
            var px = n.X >= 0 ? box.Max.X : box.Min.X;
            var py = n.Y >= 0 ? box.Max.Y : box.Min.Y;
            var pz = n.Z >= 0 ? box.Max.Z : box.Min.Z;
            if (px * n.X + py * n.Y + pz * n.Z + p.D < 0)
                return false;
        }
        return true;
    }
    public void RenderFrame()
    {
        // _renderPipeline.DrawImmediate(_cameraState);
        _renderPipeline.Draw(_cameraState);
    }
    private void SyncToolbarState(ViewportToolbar toolbar)
    {
        toolbar?.UpdateDimension(Width, Height);
    }
}
public class ArcBallController
{
    public float Sensitivity { get; set; } = 1.0f;
    public Quaternion CalcRotation(Vector2 start, Vector2 end, int viewportW, int viewportH)
    {
        var from = MapToSphere(start, viewportW, viewportH);
        var to = MapToSphere(end, viewportW, viewportH);
        var axis = Vector3.Cross(from, to);
        var dot = Vector3.Dot(from, to);
        if (axis.LengthSquared() < 1e-12f)
            return Quaternion.Identity;
        var angle = MathF.Acos((float)Math.Clamp(dot, -1.0, 1.0)) * Sensitivity;
        return Quaternion.CreateFromAxisAngle(Vector3.Normalize(axis), angle);
    }
    private static Vector3 MapToSphere(Vector2 screen, int w, int h)
    {
        // 屏幕 Y 轴向下，翻转后才能与 NDC 一致
        float x = (2.0f * screen.X / w) - 1.0f;
        float y = 1.0f - (2.0f * screen.Y / h);
        float z2 = 1.0f - x * x - y * y;
        float z = z2 > 0 ? MathF.Sqrt(z2) : 0.0f;
        var v = new Vector3(x, y, z);
        var len = v.Length();
        return len > 1e-6f ? v / len : Vector3.UnitZ;
```

```
        }
        public Matrix4x4 ApplyToCamera(Quaternion delta, CameraState camera)
        {
            camera.Orientation = Quaternion.Normalize(delta * camera.Orientation);
            var rot = Matrix4x4.CreateFromQuaternion(camera.Orientation);
            var trans = Matrix4x4.CreateTranslation(-camera.Position);
            camera.ViewMatrix = trans * rot;
            return camera.ViewMatrix;
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
namespace Origami.CreaseTo3D
{
    public class FacePicker
    {
        private const float EPSILON = 1e-6f;
        /// <summary>
        /// 从屏幕坐标拾取最近的三维面片。
        /// </summary>
        public (int faceId, Vector3 hitPoint)? PickFace(
            int screenX, int screenY,
            Matrix4x4 invProj,
            IEnumerable<(int faceId, Vector3 v0, Vector3 v1, Vector3 v2)> faceTris,
            Vector2 viewport)
        {
            if (viewport.X <= 0 || viewport.Y <= 0)
                return null;
            float ndcX = 2.0f * screenX / viewport.X - 1.0f;
            float ndcY = 1.0f - 2.0f * screenY / viewport.Y;
            Vector4 nearClip = new Vector4(ndcX, ndcY, 0.0f, 1.0f);
            Vector4 farClip = new Vector4(ndcX, ndcY, 1.0f, 1.0f);
            // 注意：这里使用逆投影矩阵而非相机参数，以兼容正交/透视两种视口
            Vector4 nearWorld = Vector4.Transform(nearClip, invProj);
            Vector4 farWorld = Vector4.Transform(farClip, invProj);
            nearWorld /= nearWorld.W;
            farWorld /= farWorld.W;
            Vector3 rayOrigin = new Vector3(nearWorld.X, nearWorld.Y, nearWorld.Z);
            Vector3 rayDir = Vector3.Normalize(new Vector3(farWorld.X, farWorld.Y, farWorld.Z) - rayOrigin);
            // 旧实现：基于固定相机方向，无法处理正交/透视混合视口
            // Vector3 rayDir = Vector3.Normalize(_cameraLookAt - rayOrigin);
            int pickedFaceId = -1;
            Vector3 nearestHit = Vector3.Zero;
            float minDistance = float.MaxValue;
            bool hasHit = false;
            foreach (var tri in faceTris)
            {
```

```
            if (!MollerTrumboreIntersect(rayOrigin, rayDir, tri.v0, tri.v1, tri.v2, out float t, out Vector3 hitPoint))
                continue;
            if (t > EPSILON && t < minDistance)
            {
                minDistance = t;
                pickedFaceId = tri.faceId;
                nearestHit = hitPoint;
                hasHit = true;
            }
        }
        if (!hasHit) return null;
        return (pickedFaceId, nearestHit);
    }
    private bool MollerTrumboreIntersect(
        Vector3 orig, Vector3 dir,
        Vector3 v0, Vector3 v1, Vector3 v2,
        out float t, out Vector3 hitPoint)
    {
        t = 0;
        hitPoint = Vector3.Zero;
        Vector3 edge1 = v1 - v0;
        Vector3 edge2 = v2 - v0;
        Vector3 pvec = Vector3.Cross(dir, edge2);
        float det = Vector3.Dot(edge1, pvec);
        if (Math.Abs(det) < EPSILON)
            return false;
        float invDet = 1.0f / det;
        Vector3 tvec = orig - v0;
        float u = invDet * Vector3.Dot(tvec, pvec);
        if (u < 0.0f || u > 1.0f)
            return false;
        Vector3 qvec = Vector3.Cross(tvec, edge1);
        float v = invDet * Vector3.Dot(dir, qvec);
        if (v < 0.0f || u + v > 1.0f)
            return false;
        t = invDet * Vector3.Dot(edge2, qvec);
        if (t > EPSILON)
        {
            hitPoint = orig + dir * t;
            return true;
        }
        return false;
    }
}
public class ParamInspectorPanel
{
    private readonly FacePicker _picker;
    private int _activeFaceId = -1;
    public ParamInspectorPanel()
    {
```

```
        _picker = new FacePicker();
    }
    public (int faceId, Vector3 hitPoint)? PickFaceFromScreen(
        int mouseX, int mouseY,
        Matrix4x4 invProj,
        IEnumerable<(int faceId, Vector3 v0, Vector3 v1, Vector3 v2)> faceTris,
        Vector2 viewport)
    {
        var hit = _picker.PickFace(mouseX, mouseY, invProj, faceTris, viewport);
        if (hit.HasValue)
            _activeFaceId = hit.Value.faceId;
        return hit;
    }
    /// <summary>
    /// 修改边长或角度后，通过共享边约束消解相邻面片顶点冲突。
    /// </summary>
    public Dictionary<int, Vector3> AdjustSharedEdgeLength(
        List<Vector3> adjVerts,
        List<(int idxA, int idxB, float targetLen)> edgeTargets,
        List<int> pinIds)
    {
        if (adjVerts == null || adjVerts.Count == 0 || edgeTargets == null || edgeTargets.Count == 0)
            return new Dictionary<int, Vector3>();
        var verts = adjVerts.ToList();
        var pins = new HashSet<int>(pinIds ?? new List<int>());
        // 高斯-牛顿迭代：将共享边长度约束线性化后用最小二乘求解
        for (int iter = 0; iter < 20; iter++)
        {
            int n = verts.Count;
            var ATA = new float[n * 3, n * 3];
            var ATb = new float[n * 3];
            foreach (var (i, j, targetLen) in edgeTargets)
            {
                Vector3 diff = verts[i] - verts[j];
                float edge_len = diff.Length();
                if (edge_len < EPSILON) continue;
                Vector3 grad = diff / edge_len;
                float residual = targetLen - edge_len;
                for (int d = 0; d < 3; d++)
                {
                    ATb[i * 3 + d] += grad[d] * residual;
                    ATb[j * 3 + d] -= grad[d] * residual;
                }
                for (int d = 0; d < 3; d++)
                {
                    ATA[i * 3 + d, i * 3 + d] += grad[d] * grad[d];
                    ATA[j * 3 + d, j * 3 + d] += grad[d] * grad[d];
                    ATA[i * 3 + d, j * 3 + d] -= grad[d] * grad[d];
                    ATA[j * 3 + d, i * 3 + d] -= grad[d] * grad[d];
                }
```

```
            }
            for (int k = 0; k < n * 3; k++)
                ATA[k, k] += 0.01f;
            foreach (int p in pins)
            {
                if (p < 0 || p >= n) continue;
                for (int d = 0; d < 3; d++)
                {
                    int row = p * 3 + d;
                    for (int c = 0; c < n * 3; c++)
                        ATA[row, c] = 0;
                    ATA[row, row] = 1.0f;
                    ATb[row] = 0;
                }
            }
            var delta = SolveDenseSystem(ATA, ATb);
            for (int v = 0; v < n; v++)
            {
                verts[v] += new Vector3(delta[v * 3], delta[v * 3 + 1], delta[v * 3 + 2]);
            }
        }
        var result = new Dictionary<int, Vector3>();
        for (int i = 0; i < verts.Count; i++)
            result[i] = verts[i];
        return result;
    }
    public float RecalcFaceArea(Vector3 a, Vector3 b, Vector3 c) =>
        Vector3.Cross(b - a, c - a).Length() * 0.5f;
    private float[] SolveDenseSystem(float[,] A, float[] b)
    {
        int n = b.Length;
        var M = new float[n, n + 1];
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
                M[i, j] = A[i, j];
            M[i, n] = b[i];
        }
        for (int col = 0; col < n; col++)
        {
            int pivotRow = col;
            float pivotMax = Math.Abs(M[col, col]);
            for (int r = col + 1; r < n; r++)
            {
                if (Math.Abs(M[r, col]) > pivotMax)
                {
                    pivotMax = Math.Abs(M[r, col]);
                    pivotRow = r;
                }
            }
```

```
            if (pivotMax < EPSILON) continue;
            if (pivotRow != col)
            {
                for (int c = 0; c <= n; c++)
                {
                    float tmp = M[col, c];
                    M[col, c] = M[pivotRow, c];
                    M[pivotRow, c] = tmp;
                }
            }
            for (int r = 0; r < n; r++)
            {
                if (r == col) continue;
                float factor = M[r, col] / M[col, col];
                for (int c = col; c <= n; c++)
                    M[r, c] -= factor * M[col, c];
            }
        }
        var x = new float[n];
        for (int i = 0; i < n; i++)
            x[i] = Math.Abs(M[i, i]) < EPSILON ? 0.0f : M[i, n] / M[i, i];
        return x;
    }
    private const float EPSILON = 1e-6f;
  }
}
using System;
using System.Collections.Generic;
using System.Linq;
namespace Origami.CreaseTo3D
{
  public enum CreaseType { Mountain, Valley, Boundary }
  public enum FoldDirection { Mountain, Valley, Flat }
  public enum ParamSource { Manual, Inferred, Default }
  public class FaceCluster
  {
    public FaceCluster()
    {
    }
    public ClusterResult ClusterByFoldingPattern(
      IReadOnlyList<(int faceId, double[] foldAngles, int edgeCount, CreaseType[] creaseSeq)> faces,
      double similarityThreshold = 0.92)
    {
      if (faces == null || faces.Count == 0)
        throw new ArgumentException("面片集合不能为空", nameof(faces));
      if (similarityThreshold <= 0.0 || similarityThreshold > 1.0)
        similarityThreshold = 0.92;
      var featureVectors = faces.Select(ExtractFeatureVector).ToArray();
      var labels = HierarchicalCluster(featureVectors, similarityThreshold);
      var clusters = faces
```

```
            .Select((f, i) => (f.faceId, label: labels[i]))
            .GroupBy(x => x.label)
            .ToDictionary(
                g => g.Key,
                g => (IReadOnlyList<int>)g.Select(x => x.faceId).ToList());
        return new ClusterResult { Labels = labels, Clusters = clusters };
    }
    private double[] ExtractFeatureVector(
        (int faceId, double[] foldAngles, int edgeCount, CreaseType[] creaseSeq) face)
    {
        var creaseBits = face.creaseSeq
            .SelectMany(t => new[]
            {
                t == CreaseType.Mountain ? 1.0 : 0.0,
                t == CreaseType.Valley ? 1.0 : 0.0,
                t == CreaseType.Boundary ? 1.0 : 0.0
            })
            .ToArray();
        var angleStats = new[]
        {
            face.foldAngles.Any() ? face.foldAngles.Average() : 0.0,
            face.foldAngles.Any() ? face.foldAngles.Max() - face.foldAngles.Min() : 0.0,
            face.edgeCount / 10.0
        };
        return angleStats.Concat(creaseBits).ToArray();
    }
    private int[] HierarchicalCluster(double[][] features, double threshold)
    {
        int n = features.Length;
        var parent = Enumerable.Range(0, n).ToArray();
        var similarities = Enumerable.Range(0, n)
            .SelectMany(i => Enumerable.Range(i + 1, n - i - 1), (i, j) => (i, j, sim: CosineSimilarity(features[i], features[j])))
            .Where(t => t.sim >= threshold)
            .OrderByDescending(t => t.sim)
            .ToList();
        foreach (var (i, j, _) in similarities)
        {
            int ri = FindRoot(parent, i);
            int rj = FindRoot(parent, j);
            if (ri != rj)
                parent[ri] = rj;
        }
        return parent.Select((_, i) => FindRoot(parent, i)).ToArray();
    }
    private int FindRoot(int[] parent, int x)
    {
        while (parent[x] != x)
        {
            parent[x] = parent[parent[x]];
            x = parent[x];
```

```
        }
        return x;
    }
    private double CosineSimilarity(double[] a, double[] b)
    {
        int dim = Math.Min(a.Length, b.Length);
        double dot = 0.0, normA = 0.0, normB = 0.0;
        for (int k = 0; k < dim; k++)
        {
            dot += a[k] * b[k];
            normA += a[k] * a[k];
            normB += b[k] * b[k];
        }
        double denom = Math.Sqrt(normA) * Math.Sqrt(normB);
        return denom < 1e-9 ? 0.0 : dot / denom;
    }
    public (double angle, FoldDirection direction, double confidence) WeightedVoting(
        IReadOnlyList<(double foldAngle, FoldDirection direction, ParamSource source)> faceParams)
    {
        if (faceParams == null || faceParams.Count == 0)
            throw new ArgumentException("同簇面片参数集合不能为空", nameof(faceParams));
        // 旧实现：简单平均，未区分手动/推断/默认权重
        // double avgAngle = faceParams.Average(p => p.foldAngle);
        var weights = faceParams.Select(p => GetSourceWeight(p.source)).ToArray();
        double totalWeight = weights.Sum();
        if (totalWeight < 1e-9) totalWeight = 1.0;
        double weightedAngle = faceParams
            .Select((p, i) => p.foldAngle * weights[i])
            .Sum() / totalWeight;
        var directionVotes = faceParams
            .GroupBy(p => p.direction)
            .Select(g => new { dir = g.Key, w = g.Sum(p => GetSourceWeight(p.source)) })
            .OrderByDescending(x => x.w)
            .ToList();
        var dominant = directionVotes.First();
        double outlierRatio = DetectOutlierRatio(faceParams, weightedAngle);
        double confidence = (dominant.w / totalWeight) * (1.0 - outlierRatio);
        if (outlierRatio > 0.35)
        {
            var downgraded_params = faceParams
                .Select(p => (p.foldAngle, p.direction, source: ParamSource.Inferred))
                .ToList();
            return WeightedVoting(downgraded_params);
        }
        return (weightedAngle, dominant.dir, confidence);
    }
    private double GetSourceWeight(ParamSource source) => source switch
    {
        ParamSource.Manual => 1.0,
        ParamSource.Inferred => 0.6,
```

```
        ParamSource.Default => 0.2,
        _ => 0.2
    };
    private double DetectOutlierRatio(
        IReadOnlyList<(double foldAngle, FoldDirection direction, ParamSource source)> faceParams,
        double centerAngle)
    {
        if (faceParams.Count < 4) return 0.0;
        var deviations = faceParams
            .Select(p => Math.Abs(p.foldAngle - centerAngle))
            .OrderBy(d => d)
            .ToArray();
        double mad = deviations[deviations.Length / 2];
        double outlier_threshold = Math.Max(mad * 2.5, 3.0);
        return (double)deviations.Count(d => d > outlier_threshold) / deviations.Length;
    }
}
public class BatchApplyDialog
{
    private readonly FaceCluster _cluster;
    public BatchApplyDialog()
    {
        _cluster = new FaceCluster();
    }
    public ClusterResult PreviewClusterResult(
        IReadOnlyList<(int faceId, double[] foldAngles, int edgeCount, CreaseType[] creaseSeq)> faces)
    {
        return _cluster.ClusterByFoldingPattern(faces, 0.92);
    }
    public SyncParamRequest BuildSyncRequest(
        IReadOnlyList<(int faceId, double[] foldAngles, int edgeCount, CreaseType[] creaseSeq)> faces,
        IReadOnlyDictionary<int, IReadOnlyList<(double angle, FoldDirection dir, ParamSource src)>> paramMap)
    {
        var clusters = _cluster.ClusterByFoldingPattern(faces);
        var request = new SyncParamRequest();
        foreach (var kv in clusters.Clusters)
        {
            var candidates = kv.Value
                .Where(paramMap.ContainsKey)
                .SelectMany(fid => paramMap[fid])
                .ToList();
            if (candidates.Count == 0) continue;
            var (angle, dir, confidence) = _cluster.WeightedVoting(candidates);
            request.AddCluster(kv.Value, angle, dir, confidence);
        }
        return request;
    }
    public IReadOnlyList<int> FilterCandidatesByCreaseLayers(
        IReadOnlyList<(int faceId, CreaseType[] creases)> faces,
        IReadOnlyCollection<CreaseType> activeLayers)
```

V1.0

```
        {
            if (activeLayers == null || activeLayers.Count == 0)
                return faces.Select(f => f.faceId).ToList();
            return faces
                .Where(f => f.creases.Any(activeLayers.Contains))
                .Select(f => f.faceId)
                .ToList();
        }
        public bool CanBatchApplyForPattern(string patternCode, int faceCount)
        {
            return !string.IsNullOrWhiteSpace(patternCode)
                && patternCode.StartsWith("CP")
                && faceCount >= 2;
        }
        public bool ValidateEntries(
            IReadOnlyList<(IReadOnlyList<int> faceIds, double angle, FoldDirection dir, double confidence)> entries)
        {
            try
            {
                return entries != null
                    && entries.Count > 0
                    && entries.All(e =>
                        e.faceIds != null && e.faceIds.Count > 0
                        && e.confidence >= 0.0 && e.confidence <= 1.0);
            }
            catch
            {
                return false;
            }
        }
    }
}
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Numerics;
namespace Origami.CreaseTo3D
{
    public class ObjSerializer
    {
        private const string ObjHeader = "# CreaseTo3D OBJ export";
        /// <summary>
        /// 将三维折纸面片序列化为 OBJ 文件，含单位缩放、坐标精度舍入与法线生成。
        /// </summary>
        public string Serialize(IEnumerable<OrigamiFace> faces, LengthUnit unit, int precision, AxisMapping axis, string path)
        {
            if (faces == null) throw new ArgumentNullException(nameof(faces));
            if (string.IsNullOrWhiteSpace(path)) throw new ArgumentNullException(nameof(path));
```

```
        if (precision < 0 || precision > 9) precision = 6;
        double scale = UnitScale(unit);
        string fmt = "F" + precision;
        var verts = new List<Vector3>();
        var normals = new List<Vector3>();
        var facesOut = new List<(int a, int b, int c, int n)>();
        var idxMap = new Dictionary<string, int>();
        foreach (var f in faces.Where(x => x != null && x.Vertices.Count >= 3))
        {
            var tri = f.Vertices
                .Take(3)
                .Select(v => Round(axis.Transform(v) * (float)scale, precision))
                .ToArray();
            int[] vi = new int[3];
            for (int i = 0; i < 3; i++)
            {
                string key = string.Format(CultureInfo.InvariantCulture, "{0}:{1}:{2}", tri[i].X, tri[i].Y, tri[i].Z);
                if (!idxMap.TryGetValue(key, out vi[i]))
                {
                    vi[i] = verts.Count + 1;
                    idxMap[key] = vi[i];
                    verts.Add(tri[i]);
                }
            }
            var n = Normal(tri[0], tri[1], tri[2]);
            facesOut.Add((vi[0], vi[1], vi[2], normals.Count + 1));
            normals.Add(n);
        }
        try
        {
            WriteObj(path, verts, normals, facesOut, fmt);
        }
        catch (IOException)
        {
            string fallback_path = Path.Combine(Path.GetTempPath(), Path.GetFileName(path));
            WriteObj(fallback_path, verts, normals, facesOut, fmt);
            path = fallback_path;
        }
        return path;
    }
    private static void WriteObj(string path, List<Vector3> verts, List<Vector3> normals, List<(int a, int b, int c, int n)> faces,
string fmt)
    {
        using var w = new StreamWriter(path, false, System.Text.Encoding.UTF8);
        w.WriteLine(ObjHeader);
        foreach (var v in verts)
            w.WriteLine("v {0} {1} {2}",
                v.X.ToString(fmt, CultureInfo.InvariantCulture),
                v.Y.ToString(fmt, CultureInfo.InvariantCulture),
                v.Z.ToString(fmt, CultureInfo.InvariantCulture));
```

V1.0

```
        foreach (var n in normals)
            w.WriteLine("vn {0} {1} {2}",
                n.X.ToString("F6", CultureInfo.InvariantCulture),
                n.Y.ToString("F6", CultureInfo.InvariantCulture),
                n.Z.ToString("F6", CultureInfo.InvariantCulture));
        foreach (var f in faces)
            w.WriteLine("f {0}//{3} {1}//{3} {2}//{3}", f.a, f.b, f.c, f.n);
    }
    private static double UnitScale(LengthUnit unit) => unit switch
    {
        LengthUnit.Millimeter => 1.0,
        LengthUnit.Centimeter => 10.0,
        LengthUnit.Meter => 1000.0,
        LengthUnit.Inch => 25.4,
        _ => 1.0
    };
    private static Vector3 Round(Vector3 v, int p)
    {
        double pow = Math.Pow(10, p);
        return new Vector3(
            (float)(Math.Round(v.X * pow) / pow),
            (float)(Math.Round(v.Y * pow) / pow),
            (float)(Math.Round(v.Z * pow) / pow));
    }
    private static Vector3 Normal(Vector3 a, Vector3 b, Vector3 c)
    {
        var n = Vector3.Cross(b - a, c - a);
        return n.LengthSquared() < 1e-10f ? Vector3.UnitZ : Vector3.Normalize(n);
    }
}
public class ExportJobPanel
{
    private readonly ObjSerializer _objSerializer;
    public ExportJobPanel()
    {
        _objSerializer = new ObjSerializer();
    }
    /// <summary>
    /// 按加权评分对导出任务排序并执行最高分任务，失败则降级后台重试。
    /// </summary>
    public IReadOnlyList<ExportTask> ScheduleTasks(IEnumerable<ExportTask> queue, ResourceOccupancy resource)
    {
        if (queue == null) return Array.Empty<ExportTask>();
        var ordered = queue
            .Where(t => t != null && t.Status == TaskStatus.Pending)
            .Select(t => new { Task = t, Score = Score(t, resource) })
            .OrderByDescending(x => x.Score)
            .Select(x => x.Task)
            .ToList();
        if (ordered.Count == 0) return ordered;
```

```
        var first = ordered[0];
        if (resource.HasAvailableSlot(first.Format))
        {
            try
            {
                Dispatch(first);
                first.Status = TaskStatus.Completed;
            }
            catch (Exception)
            {
                // 失败不阻塞整个队列，降级到后台重试并降低优先级避免持续抢占
                first.Status = TaskStatus.BackgroundRetry;
                first.PriorityLevel = Math.Max(0, first.PriorityLevel - 1);
            }
        }
        else
        {
            first.WaitTime += TimeSpan.FromMinutes(1);
        }
        return ordered;
    }
    private double Score(ExportTask t, ResourceOccupancy r)
    {
        // 旧公式：return t.PriorityLevel;  // 导致大文件/复杂格式长期饥饿，2024-02 改为加权
        double complexity = t.Format switch
        {
            ExportFormat.Obj => 1.0,
            ExportFormat.Stl => 1.2,
            ExportFormat.Dxf => 1.5,
            ExportFormat.Svg => 0.8,
            _ => 1.0
        };
        double wait = t.WaitTime.TotalMinutes;
        double s = complexity * 0.35 + wait * 0.25 + t.PriorityLevel * 0.4;
        return r.HasAvailableSlot(t.Format) ? s : s * 0.7;
    }
    private void Dispatch(ExportTask t)
    {
        if (t.Format == ExportFormat.Obj)
        {
            t.OutputPath = _objSerializer.Serialize(t.Faces, t.TargetUnit, t.Precision, t.Axis, t.OutputPath);
        }
        else
        {
            throw new NotSupportedException("Sibling serializer handles " + t.Format);
        }
    }
  }
}
using System;
```

```
using System.Collections.Generic;
using System.IO;
using System.IO.Compression;
using System.Linq;
using System.Security.Cryptography;
using System.Text;
namespace Origami.CreaseTo3D
{
  public class VersionSnapshot
  {
    public Guid SnapshotId { get; set; }
    public string ModelName { get; set; }
    public byte[] RawState { get; set; }
    public string ManifestHash { get; set; }
    public DateTime CreatedAt { get; set; }
    public List<string> SelectedHalfEdges { get; set; } = new List<string>();
    public string SortMode { get; set; } = "按拓扑 ID 升序";
    public Dictionary<string, List<string>> TopologyGraph { get; set; } = new Dictionary<string, List<string>>();
  }
  public class ArchivePanel
  {
    public ArchivePanel()
    {
    }
    public VersionSnapshot BatchSelectHalfEdges(VersionSnapshot snap, string mode, List<string> candidates)
    {
      if (snap == null || candidates == null) return snap;
      var selected = new List<string>();
      switch (mode)
      {
        case "全选":
          selected = candidates.ToList();
          break;
        case "仅选边界":
          selected = candidates.Where(he => IsBoundaryHalfEdge(he, snap.TopologyGraph)).ToList();
          break;
        case "全不选":
          selected.Clear();
          break;
        default:
          selected = candidates.ToList();
          break;
      }
      snap.SelectedHalfEdges = selected;
      return snap;
    }
    private bool IsBoundaryHalfEdge(string half_edge_id, Dictionary<string, List<string>> graph)
    {
      // 2024-03: 边界半边判定改为依赖拓扑图而非字符串前缀，避免 V08/V09 这类顶点 ID 被误判
      if (!graph.ContainsKey(half_edge_id)) return true;
```

```
        return graph[half_edge_id].Count < 2;
    }
    public List<string> SortTopology(VersionSnapshot snap, List<string> ids, string sortMode)
    {
        snap.SortMode = sortMode;
        var list = ids.ToList();
        switch (sortMode)
        {
            case "按拓扑 ID 升序":
                return list.OrderBy(id => id, StringComparer.Ordinal).ToList();
            case "按连接数降序":
                return list.OrderByDescending(id => GetConnectionCount(id, snap.TopologyGraph)).ToList();
            case "按类型分组":
                return list
                    .GroupBy(id => id.StartsWith("V") ? "顶点" : (id.StartsWith("HE") ? "半边" : "面"))
                    .SelectMany(g => g)
                    .ToList();
            default:
                // old: return list.OrderBy(id => id.Length).ToList();
                return list;
        }
    }
    private int GetConnectionCount(string id, Dictionary<string, List<string>> graph)
    {
        return graph.ContainsKey(id) ? graph[id].Count : 0;
    }
    public byte[] CreateDeltaArchive(VersionSnapshot current, VersionSnapshot baseline)
    {
        if (current?.RawState == null || baseline?.RawState == null)
            throw new ArgumentException("snapshot state missing");
        var diffBlocks = DiffStates(current.RawState, baseline.RawState);
        var payload = SerializeDiffBlocks(diffBlocks);
        using (var ms = new MemoryStream())
        {
            // old impl: using (var gzip = new GZipStream(ms, CompressionLevel.Optimal)) { gzip.Write(payload, 0,
payload.Length); }
            using (var zlib = new ZLibStream(ms, CompressionLevel.Optimal))
            {
                zlib.Write(payload, 0, payload.Length);
            }
            return ms.ToArray();
        }
    }
    private List<(int offset, byte[] data)> DiffStates(byte[] current, byte[] baseline)
    {
        var blocks = new List<(int, byte[])>();
        int len = Math.Min(current.Length, baseline.Length);
        int i = 0;
        while (i < len)
        {
```

```
            if (current[i] != baseline[i])
            {
                int start = i;
                while (i < len && current[i] != baseline[i]) i++;
                int blockLen = i - start;
                var block = new byte[blockLen];
                Buffer.BlockCopy(current, start, block, 0, blockLen);
                blocks.Add((start, block));
            }
            else
            {
                i++;
            }
        }
        if (current.Length > baseline.Length)
        {
            var tail = new byte[current.Length - baseline.Length];
            Buffer.BlockCopy(current, baseline.Length, tail, 0, tail.Length);
            blocks.Add((baseline.Length, tail));
        }
        return blocks;
    }
    private byte[] SerializeDiffBlocks(List<(int offset, byte[] data)> blocks)
    {
        using (var ms = new MemoryStream())
        using (var writer = new BinaryWriter(ms, Encoding.UTF8))
        {
            writer.Write(blocks.Count);
            foreach (var (offset, data) in blocks)
            {
                writer.Write(offset);
                writer.Write(data.Length);
                writer.Write(data);
            }
            return ms.ToArray();
        }
    }
    public bool VerifyArchiveIntegrity(byte[] archivePackage, string expectedSha256)
    {
        if (archivePackage == null || archivePackage.Length == 0 || string.IsNullOrWhiteSpace(expectedSha256))
            return false;
        using (var sha = SHA256.Create())
        {
            var hash = sha.ComputeHash(archivePackage);
            var actual = BitConverter.ToString(hash).Replace("-", "").ToLowerInvariant();
            // 清单摘要统一转小写，防止 Windows/Linux 导出大小写不一致导致误判
            return actual == expectedSha256.ToLowerInvariant();
        }
    }
}
```

```
}
using System;
using System.Collections.Generic;
using System.Linq;
namespace Origami.CreaseTo3D
{
    public class NotifyHub
    {
        private readonly NotifyDispatcher _dispatcher;
        public NotifyHub()
        {
            _dispatcher = new NotifyDispatcher();
        }
        /// <summary>
        /// 状态事件聚合与节流：窗口内同类型事件只保留最新一条。
        /// </summary>
        public IReadOnlyList<StatusEvent> ThrottleStatusEvents(IEnumerable<StatusEvent> eventStream, TimeSpan window)
        {
            if (eventStream == null)
                return new List<StatusEvent>();
            // 窗口不能小于一次 UI 刷新周期，否则高频事件会全丢
            if (window < TimeSpan.FromMilliseconds(50))
                window = TimeSpan.FromMilliseconds(50);
            var cutoff = DateTime.UtcNow - window;
            var merged = eventStream
                .Where(e => e != null && e.IsValid && e.Timestamp >= cutoff)
                .GroupBy(e => e.EventType)
                .Select(g => g
                    .OrderByDescending(e => e.Timestamp)
                    .ThenByDescending(e => e.SequenceId)
                    .First())
                .Where(e => !string.IsNullOrEmpty(e.Payload))
                .OrderBy(e => e.Timestamp)
                .ToList();
            // 旧实现：直接 Distinct，无法处理同类型多次更新，保留作对比
            // var legacy = eventStream.Distinct().ToList();
            try
            {
                _dispatcher.Dispatch(merged);
            }
            catch
            {
                // 通知分发失败不应阻塞状态聚合
            }
            return merged;
        }
        /// <summary>
        /// 系统日志级别自适应过滤：按模块相关度与级别加权评分。
        /// </summary>
        public IReadOnlyList<SystemLogSink> AdaptiveFilterLogs(
```

V1.0
```
        IEnumerable<SystemLogSink> logs,
        string focusModule,
        LogLevelFilter baseThreshold)
    {
        if (logs == null)
            return new List<SystemLogSink>();
        int baseScore = (int)baseThreshold;
        int dynamicThreshold = Math.Max(0, baseScore - ModuleBoost(focusModule));
        // TODO: 当日志流超过 1k 条/秒时，此处应换为滑动窗口堆
        var scored = logs
            .Where(l => l != null)
            .Select(l => new { Log = l, Score = CalcLogScore(l, focusModule) })
            .Where(x => x.Score >= dynamicThreshold)
            .OrderByDescending(x => x.Score)
            .ThenByDescending(x => x.Log.Timestamp)
            .ToList();
        return scored.Select(x => x.Log).ToList();
    }
    int CalcLogScore(SystemLogSink log, string focusModule)
    {
        int levelScore = (int)log.Level * 2;
        int relevance = ModuleRelevance(log.Module, focusModule);
        int recency = log.Timestamp > DateTime.UtcNow.AddMinutes(-5) ? 1 : 0;
        return levelScore + relevance * 5 + recency;
    }
    int ModuleRelevance(string logModule, string focusModule)
    {
        if (string.IsNullOrEmpty(focusModule) || string.IsNullOrEmpty(logModule))
            return 0;
        if (string.Equals(logModule, focusModule, StringComparison.OrdinalIgnoreCase))
            return 3;
        if (logModule.StartsWith(focusModule, StringComparison.OrdinalIgnoreCase))
            return 2;
        if (logModule.EndsWith(focusModule, StringComparison.OrdinalIgnoreCase))
            return 1;
        return 0;
    }
    int ModuleBoost(string focus_module)
    {
        return string.IsNullOrEmpty(focus_module) ? 0 : 1;
    }
    // 预留：批量刷新挂起通知，后续与前端 SignalR 通道对接
    void FlushPending(IReadOnlyList<StatusEvent> pending)
    {
        if (pending.Count == 0) return;
        _dispatcher.Dispatch(pending);
    }
}
public class StatusIndicatorBar
{
```

```
    private readonly NotifyHub _hub;
    public string CurrentState { get; private set; }
    public string CreasePatternState { get; private set; }
    public string TopologyState { get; private set; }
    public string ModelState { get; private set; }
    public StatusIndicatorBar(NotifyHub hub)
    {
        _hub = hub ?? new NotifyHub();
        Reset();
    }
    public void UpdateFromEvents(IEnumerable<StatusEvent> events)
    {
        if (events == null) return;
        var throttled = _hub.ThrottleStatusEvents(events, TimeSpan.FromSeconds(1));
        foreach (var e in throttled)
        {
            switch (e.EventType)
            {
                case "SystemReady":
                    CurrentState = "系统就绪";
                    break;
                case "CreasePatternLoaded":
                    CreasePatternState = "折痕图已加载";
                    break;
                case "TopologyBuilt":
                    TopologyState = "拓扑已构建";
                    break;
                case "ModelGenerated":
                    ModelState = "模型已生成";
                    break;
                case "ValidationWarning":
                    // 2024-05 后产品要求：警告态保留前序成功态，避免状态栏闪回
                    CurrentState = $"系统警告: {e.Payload}";
                    break;
            }
        }
    }
    public void Reset()
    {
        CurrentState = "系统就绪";
        CreasePatternState = "折痕图未加载";
        TopologyState = "拓扑未构建";
        ModelState = "模型未生成";
    }
  }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
```

```
using System.Text;
namespace Origami.CreaseTo3D
{
    public sealed class PasswordHasher
    {
        private const int SaltBytes = 16;
        private const int HashBytes = 32;
        private const int Iterations = 600000;
        public static string Hash(string pwd)
        {
            if (string.IsNullOrWhiteSpace(pwd))
                throw new ArgumentException("密码不能为空", nameof(pwd));
            var salt = new byte[SaltBytes];
            using (var rng = RandomNumberGenerator.Create())
                rng.GetBytes(salt);
            var hash = Pbkdf2(pwd, salt, Iterations, HashBytes);
            return $"{Iterations}:{Convert.ToBase64String(salt)}:{Convert.ToBase64String(hash)}";
        }
        public static bool Verify(string pwd, string stored)
        {
            if (string.IsNullOrWhiteSpace(pwd) || string.IsNullOrWhiteSpace(stored))
                return false;
            try
            {
                var parts = stored.Split(':');
                if (parts.Length != 3)
                    return false;
                var iter = int.Parse(parts[0]);
                var salt = Convert.FromBase64String(parts[1]);
                var expected = Convert.FromBase64String(parts[2]);
                var actual = Pbkdf2(pwd, salt, iter, expected.Length);
                return CryptographicOperations.FixedTimeEquals(actual, expected);
            }
            catch
            {
                // 解析失败统一视为校验不通过，避免泄露存储结构
                return false;
            }
        }
        public static bool NeedsRehash(string stored, int targetIterations = Iterations)
        {
            if (string.IsNullOrWhiteSpace(stored))
                return true;
            var parts = stored.Split(':');
            return parts.Length != 3
                || !int.TryParse(parts[0], out var iter)
                || iter < targetIterations;
        }
        private static byte[] Pbkdf2(string pwd, byte[] salt, int iter, int len)
        {
```

```
        // 旧实现：using var derive = new Rfc2898DeriveBytes(pwd, salt, iter);
        using var derive = new Rfc2898DeriveBytes(pwd, salt, iter, HashAlgorithmName.SHA256);
        return derive.GetBytes(len);
    }
}
public sealed class LoginResponse
{
    public string UserId { get; set; }
    public string AccountName { get; set; }
    public string AccessToken { get; set; }
    public string RefreshToken { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }
    public IReadOnlyList<string> Permissions { get; set; }
    public string SelectedDbNode { get; set; }
    public string SelectedCreasePatternSource { get; set; }
    public bool IsAlive => DateTimeOffset.UtcNow < ExpiresAt;
    public static LoginResponse Build(string userId, string accountName, IEnumerable<string> permissions, string dbNode)
    {
        if (string.IsNullOrWhiteSpace(userId) || string.IsNullOrWhiteSpace(accountName))
            throw new ArgumentException("用户标识与账户名为必填项");
        return new LoginResponse
        {
            UserId = userId,
            AccountName = accountName,
            Permissions = new List<string>(permissions ?? Array.Empty<string>()),
            SelectedDbNode = dbNode,
            ExpiresAt = DateTimeOffset.UtcNow.AddHours(8)
        };
    }
    public TimeSpan RemainingLifetime() => ExpiresAt - DateTimeOffset.UtcNow;
    public bool CanRebuildTopology()
    {
        // 2024-03-15 后，重建入口必须同时校验权限与 DB 节点绑定状态
        var perms = Permissions ?? Array.Empty<string>();
        return perms.Contains("crease:rebuild") && !string.IsNullOrEmpty(SelectedDbNode);
    }
    public string SignRebuildPayload(string sourceId)
    {
        if (string.IsNullOrWhiteSpace(sourceId))
            throw new ArgumentException("数据源标识不能为空", nameof(sourceId));
        if (!CanRebuildTopology())
            throw new InvalidOperationException("当前会话未授权重建操作");
        var _payload = $"{UserId}|{sourceId}|{SelectedDbNode}|{DateTimeOffset.UtcNow.ToUnixTimeSeconds()}";
        var key = DeriveSigningKey();
        // 旧实现：return $"{AccessToken}:{sourceId}";
        using var hmac = new HMACSHA256(key);
        var bytes = Encoding.UTF8.GetBytes(_payload);
        var sig = hmac.ComputeHash(bytes);
        return $"{Convert.ToBase64String(bytes)}.{Convert.ToBase64String(sig)}";
    }
```

V1.0
```
    public static bool TryParseCreasePatternLabel(string label, out int vCount, out int hCount, out int fCount)
    {
        vCount = hCount = fCount = 0;
        if (string.IsNullOrWhiteSpace(label))
            return false;
        try
        {
            var start = label.IndexOf('(');
            var end = label.IndexOf(')', start);
            if (start < 0 || end < 0)
                return false;
            var inner = label.Substring(start + 1, end - start - 1);
            var nums = inner.Split(new[] { '/', ' ' }, StringSplitOptions.RemoveEmptyEntries)
                            .Where(s => int.TryParse(s, out _))
                            .Select(int.Parse)
                            .ToArray();
            if (nums.Length < 3)
                return false;
            vCount = nums[0];
            hCount = nums[1];
            fCount = nums[2];
            return true;
        }
        catch
        {
            return false;
        }
    }
    private byte[] DeriveSigningKey()
    {
        // 临时派生：以 RefreshToken 前 32 字节作为对称密钥，生产环境由 KMS 轮换
        var src = Encoding.UTF8.GetBytes(RefreshToken ?? string.Empty);
        using var sha = SHA256.Create();
        return sha.ComputeHash(src);
    }
    // 后续审计日志需要，当前未调用
    private string AuditStamp() => $"{AccountName}@{SelectedDbNode ?? "unknown"}";
  }
}
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
namespace Origami.CreaseTo3D
{
  public class WorkObjectSwitchRequest
  {
    public string TargetWorkObjectId { get; set; }
    public string SourceTabId { get; set; }
    public string TriggerMode { get; set; }
```

V1.0

```
    public bool ForceSwitch { get; set; }
    public bool ValidateStateChain { get; set; } = true;
 }
 public class ProjectDraft
 {
    public string DraftId { get; set; }
    public string DraftName { get; set; }
    public string CurrentWorkObjectId { get; set; }
    public List<string> OpenedTabIds { get; set; } = new List<string>();
    public bool CreasePatternLoaded { get; set; }
    public bool TopologyBuilt { get; set; }
    public bool ModelGenerated { get; set; }
    public string TopologyValidationState { get; set; } = "未校验";
    public int TopologyErrorCount { get; set; }
    public Dictionary<string, int> LastTopologyStats { get; set; } = new Dictionary<string, int>();
    public List<string> ValidationLog { get; set; } = new List<string>();
    private const string MODE_RESTORE = "restore";
    private const string WO_TOPOLOGY = "topology";
    public ProjectDraft()
    {
    }
    /// <summary>
    /// 校验目标工作对象是否允许切换，依赖链不满足时返回 false。
    /// </summary>
    public bool CanSwitchTo(WorkObjectSwitchRequest req)
    {
        if (req == null || string.IsNullOrWhiteSpace(req.TargetWorkObjectId))
            return false;
        if (req.TargetWorkObjectId == CurrentWorkObjectId && !req.ForceSwitch)
            return false;
        if (!OpenedTabIds.Contains(req.TargetWorkObjectId))
            return false;
        if (!req.ValidateStateChain)
            return true;
        var dependency_ok = req.TargetWorkObjectId switch
        {
            "crease-pattern" => true,
            "topology" => CreasePatternLoaded,
            "face-generation" => CreasePatternLoaded && TopologyBuilt,
            "model-export" => CreasePatternLoaded && TopologyBuilt && ModelGenerated,
            _ => true
        };
        return dependency_ok;
    }
    /// <summary>
    /// 执行切换并记录日志；恢复模式会按标签顺序反推状态标志。
    /// </summary>
    public string ApplySwitch(WorkObjectSwitchRequest req)
    {
        if (!CanSwitchTo(req))
```

```
            return "rejected";
        var prev = CurrentWorkObjectId;
        CurrentWorkObjectId = req.TargetWorkObjectId;
        // 恢复会话时不能仅靠前端传标志，需按已打开标签重建
        if (req.TriggerMode == MODE_RESTORE && prev != null)
        {
            SyncStateByWorkObject(CurrentWorkObjectId);
        }
        ValidationLog.Add($"{DateTime.Now:HH:mm:ss} switch {prev} -> {req.TargetWorkObjectId}");
        return CurrentWorkObjectId;
    }
    /// <summary>
    /// 返回底部状态栏使用的四组就绪标志。
    /// </summary>
    public Dictionary<string, bool> GetWorkspaceStatus()
    {
        return new Dictionary<string, bool>
        {
            ["creasePatternLoaded"] = CreasePatternLoaded,
            ["topologyBuilt"] = TopologyBuilt,
            ["modelGenerated"] = ModelGenerated,
            ["hasActiveWorkObject"] = !string.IsNullOrEmpty(CurrentWorkObjectId)
        };
    }
    /// <summary>
    /// 根据拓扑结构树数据校验一致性，并更新拓扑构建标志。
    /// </summary>
    public bool ComputeTopologyReadiness(int vertices, int undirectedEdges, int halfEdges, int faces, int euler)
    {
        if (vertices <= 0 || undirectedEdges <= 0 || halfEdges <= 0 || faces <= 0)
            return false;
        // 旧实现：仅判断 halfEdges % 2 == 0，未处理非流形边界情况
        // var old_check = halfEdges % 2 == 0;
        if (halfEdges != undirectedEdges * 2)
            return false;
        // 欧拉示性数与 V-E+F 必须一致，不一致说明存在悬挂半边或重复面
        var euler_consistent = euler == vertices - undirectedEdges + faces;
        TopologyBuilt = euler_consistent;
        TopologyValidationState = euler_consistent ? "流形" : "非流形";
        TopologyErrorCount = euler_consistent ? 0 : Math.Abs(euler - (vertices - undirectedEdges + faces));
        SnapshotTopologyStats(vertices, undirectedEdges, halfEdges, faces, euler);
        return euler_consistent;
    }
    public void SnapshotTopologyStats(int vertices, int undirectedEdges, int halfEdges, int faces, int euler)
    {
        LastTopologyStats["顶点"] = vertices;
        LastTopologyStats["无向边"] = undirectedEdges;
        LastTopologyStats["半边"] = halfEdges;
        LastTopologyStats["面"] = faces;
        LastTopologyStats["欧拉示性数"] = euler;
```

```
        }
        /// <summary>
        /// 获取供界面标题区展示的统计摘要，如 "42 顶点 / 84 半边 / 40 面"。
        /// </summary>
        public string GetTopologySummary()
        {
            if (LastTopologyStats.Count == 0)
                return "拓扑未构建";
            return $"{LastTopologyStats["顶点"]} 顶点 / {LastTopologyStats["半边"]} 半边 / {LastTopologyStats["面"]} 面";
        }
        public bool ValidateTabClosure(string tabId)
        {
            if (string.IsNullOrWhiteSpace(tabId))
                return false;
            if (tabId == CurrentWorkObjectId)
            {
                CurrentWorkObjectId = OpenedTabIds.FirstOrDefault(t => t != tabId);
            }
            return OpenedTabIds.Remove(tabId);
        }
        private void SyncStateByWorkObject(string workObjectId)
        {
            var idx = OpenedTabIds.IndexOf(workObjectId);
            var topoIdx = OpenedTabIds.IndexOf(WO_TOPOLOGY);
            if (idx < 0 || topoIdx < 0)
            {
                TopologyBuilt = false;
                ModelGenerated = false;
                return;
            }
            CreasePatternLoaded = true;
            TopologyBuilt = idx >= topoIdx;
            ModelGenerated = idx > topoIdx;
        }
    }
}
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Linq;
using System.Text;
namespace Origami.CreaseTo3D
{
    public class CreaseLineDraft
    {
        public const int TypeBorder = 0;
        public const int TypeMountain = 1;
        public const int TypeValley = 2;
        public const int TypeFlat = 3;
```

V1.0

```
    public Guid Id { get; set; }
    public string Name { get; set; }
    public string LastError { get; set; }
    public string LastValidationSummary { get; set; }
    public Dictionary<int, (double x, double y, int vType)> Vertices { get; set; } = new();
    public List<(int a, int b, int cType)> Segments { get; set; } = new();
    public int AcquireVertex(double x, double y, double tol = 1e-6)
    {
        double t2 = tol * tol;
        foreach (var kv in Vertices)
        {
            double dx = kv.Value.x - x, dy = kv.Value.y - y;
            if (dx * dx + dy * dy <= t2) return kv.Key;
        }
        int id = Vertices.Count == 0 ? 1 : Vertices.Keys.Max() + 1;
        Vertices[id] = (x, y, 0);
        return id;
    }
    public void AddSegment(int a, int b, int cType)
    {
        if (a == b) throw new ArgumentException("退化折痕");
        if (!Vertices.ContainsKey(a) || !Vertices.ContainsKey(b)) throw new ArgumentException("顶点未注册");
        if (Segments.Any(s => (s.a == a && s.b == b) || (s.a == b && s.b == a))) return;
        Segments.Add((a, b, cType));
    }
    public void RemoveSegment(int a, int b) =>
        Segments.RemoveAll(s => (s.a == a && s.b == b) || (s.a == b && s.b == a));
    public void SetCreaseType(int a, int b, int cType)
    {
        for (int i = 0; i < Segments.Count; i++)
        {
            var s = Segments[i];
            if ((s.a == a && s.b == b) || (s.a == b && s.b == a))
            {
                Segments[i] = (s.a, s.b, cType);
                return;
            }
        }
    }
    public (bool manifoldOk, bool closedOk, bool orientationOk, string summary) ValidateTopologyRules(
        bool checkManifold, bool checkClosed, bool checkOrientation, double tol = 1e-6)
    {
        bool mOk = true, cOk = true, oOk = true;
        var sb = new StringBuilder();
        if (checkManifold || checkOrientation)
        {
            var edgeHalf = new Dictionary<(int u, int v), List<(int from, int to)>>();
            foreach (var (a, b, _) in Segments)
            {
                var key = a < b ? (a, b) : (b, a);
```

```
            if (!edgeHalf.ContainsKey(key)) edgeHalf[key] = new List<(int, int)>();
            edgeHalf[key].Add((a, b));
            edgeHalf[key].Add((b, a));
        }
        if (checkManifold)
        {
            foreach (var kv in edgeHalf)
            {
                if (kv.Value.Count != 2)
                {
                    mOk = false;
                    sb.AppendLine($"非流形边  V{kv.Key.u}-V{kv.Key.v} 关联{kv.Value.Count}条半边");
                }
            }
        }
        if (checkOrientation)
        {
            foreach (var kv in edgeHalf)
            {
                var h = kv.Value;
                if (h.Count != 2 || h[0].from != h[1].to || h[0].to != h[1].from)
                {
                    oOk = false;
                    sb.AppendLine($"方向不一致边  V{kv.Key.u}-V{kv.Key.v}");
                }
            }
        }
    }
    if (checkClosed)
    {
        var deg = new Dictionary<int, int>();
        foreach (var (a, b, t) in Segments)
        {
            if (t != TypeBorder) continue;
            deg[a] = deg.ContainsKey(a) ? deg[a] + 1 : 1;
            deg[b] = deg.ContainsKey(b) ? deg[b] + 1 : 1;
        }
        // Console.WriteLine($"debug: border vert count={deg.Count}");
        foreach (var kv in deg)
        {
            if (kv.Value == 1)
            {
                cOk = false;
                sb.AppendLine($"边界半边悬挂: V{kv.Key}");
            }
        }
    }
    string summary = sb.Length == 0 ? "拓扑规则校验  0 错误" : sb.ToString();
    LastValidationSummary = summary;
    return (mOk, cOk, oOk, summary);
```

```
    }
}
public class DxfEntityParser
{
    private const double SnapTol = 1e-4;
    public CreaseLineDraft Parse(Stream stream)
    {
        var draft = new CreaseLineDraft { Name = "dxf_import" };
        try
        {
            // var rawLines = File.ReadAllLines(path); // 旧实现一次性读入，大文件会爆内存
            using var reader = new StreamReader(stream);
            string code = null, entityType = null, layer = "";
            double sx = 0, sy = 0, ex = 0, ey = 0;
            var poly = new List<(double x, double y)>();
            bool polyClosed = false;
            string line;
            while ((line = reader.ReadLine()) != null)
            {
                if (code == null) { code = line.Trim(); continue; }
                string val = line.Trim();
                if (code == "0")
                {
                    FlushEntity(draft, entityType, sx, sy, ex, ey, layer, poly, polyClosed);
                    entityType = val;
                    poly = new List<(double x, double y)>();
                    polyClosed = false;
                    layer = "";
                }
                else if (entityType == "LINE")
                {
                    switch (code)
                    {
                        case "8": layer = val; break;
                        case "10": sx = ParseD(val); break;
                        case "20": sy = ParseD(val); break;
                        case "11": ex = ParseD(val); break;
                        case "21": ey = ParseD(val); break;
                    }
                }
                else if (entityType == "LWPOLYLINE")
                {
                    switch (code)
                    {
                        case "8": layer = val; break;
                        case "70": polyClosed = int.Parse(val) != 0; break;
                        case "10": poly.Add((ParseD(val), 0)); break;
                        case "20":
                            if (poly.Count > 0)
                            {
```

```
                        var last = poly[poly.Count - 1];
                        poly[poly.Count - 1] = (last.x, ParseD(val));
                    }
                    break;
                }
            }
            code = null;
        }
        FlushEntity(draft, entityType, sx, sy, ex, ey, layer, poly, polyClosed);
    }
    catch (Exception ex)
    {
        draft.LastError = ex.Message;
    }
    return draft;
}
private void FlushEntity(CreaseLineDraft draft, string entityType,
    double sx, double sy, double ex, double ey, string layer,
    List<(double x, double y)> poly, bool polyClosed)
{
    if (string.IsNullOrEmpty(entityType) || entityType == "EOF") return;
    int ctype = LayerToCreaseType(layer);
    if (entityType == "LINE")
    {
        int a = draft.AcquireVertex(sx, sy, SnapTol);
        int b = draft.AcquireVertex(ex, ey, SnapTol);
        draft.AddSegment(a, b, ctype);
    }
    else if (entityType == "LWPOLYLINE")
    {
        for (int i = 0; i + 1 < poly.Count; i++)
        {
            int a = draft.AcquireVertex(poly[i].x, poly[i].y, SnapTol);
            int b = draft.AcquireVertex(poly[i + 1].x, poly[i + 1].y, SnapTol);
            draft.AddSegment(a, b, ctype);
        }
        if (polyClosed && poly.Count > 2)
        {
            int a = draft.AcquireVertex(poly[poly.Count - 1].x, poly[poly.Count - 1].y, SnapTol);
            int b = draft.AcquireVertex(poly[0].x, poly[0].y, SnapTol);
            draft.AddSegment(a, b, ctype);
        }
    }
}
private int LayerToCreaseType(string layer)
{
    var l = (layer ?? "").ToLowerInvariant();
    if (l.Contains("border") || l.Contains("boundary") || l == "0") return CreaseLineDraft.TypeBorder;
    if (l.Contains("mountain") || l == "m") return CreaseLineDraft.TypeMountain;
    if (l.Contains("valley") || l == "v") return CreaseLineDraft.TypeValley;
```

```
            return CreaseLineDraft.TypeValley;
        }
        private double ParseD(string s) => double.Parse(s, CultureInfo.InvariantCulture);
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
namespace Origami.CreaseTo3D
{
    public class LayerVisibilityRule
    {
        private static readonly string[] _knownAttributes = { "manifold", "boundary", "nonmanifold" };
        public bool ShowVertices { get; set; } = true;
        public bool ShowFaces { get; set; } = true;
        public bool ShowNormalHalfEdges { get; set; } = true;
        public bool ShowBoundaryHalfEdges { get; set; } = true;
        public bool ShowNonManifoldElements { get; set; } = true;
        public string ActiveLayer { get; set; } = "CP001 平面网格";
        public int SeverityThreshold { get; set; } = 0;
        public LayerVisibilityRule()
        {
        }
        public bool IsVisible(string elementType, string topoAttribute, int severity = 0)
        {
            if (string.IsNullOrEmpty(elementType)) return false;
            if (severity < SeverityThreshold) return false;
            var t = elementType.ToLowerInvariant();
            var attr = (topoAttribute ?? string.Empty).ToLowerInvariant();
            return t switch
            {
                "vertex" => ShowVertices,
                "face" => ShowFaces,
                "halfedge" or "edge" => attr switch
                {
                    "boundary" => ShowBoundaryHalfEdges,
                    "nonmanifold" => ShowNonManifoldElements,
                    _ => ShowNormalHalfEdges
                },
                _ => false
            };
        }
        public bool ShouldHighlightByValidation(string validationState)
        {
            var state = (validationState ?? string.Empty).ToLowerInvariant();
            if (state.Contains("错误") || state.Contains("非流形")) return ShowNonManifoldElements;
            if (state.Contains("警告") || state.Contains("边界")) return ShowBoundaryHalfEdges;
            return false;
        }
```

```
    public int GetRenderOrder(string elementType)
    {
        return elementType?.ToLowerInvariant() switch
        {
            "face" => 0,
            "halfedge" or "edge" => 1,
            "vertex" => 2,
            _ => 99
        };
    }
    public IEnumerable<string> FilterVisibleIds(IEnumerable<string> ids, Func<string, string> typeResolver, Func<string,
string> attrResolver)
    {
        if (ids == null) yield break;
        foreach (var id in ids)
        {
            var type = typeResolver?.Invoke(id) ?? string.Empty;
            var attr = attrResolver?.Invoke(id) ?? string.Empty;
            if (IsVisible(type, attr)) yield return id;
        }
    }
}
public class StyleApplyRequest
{
    public Guid RequestId { get; set; } = Guid.NewGuid();
    public string TargetLayer { get; set; }
    public string ElementType { get; set; }
    public List<string> ElementIds { get; set; } = new List<string>();
    public string StrokeColor { get; set; }
    public string FillColor { get; set; }
    public double StrokeWidth { get; set; } = 1.0;
    public int ZIndex { get; set; }
    public string MarkerShape { get; set; }
    public bool Validate()
    {
        if (string.IsNullOrWhiteSpace(TargetLayer)) return false;
        if (string.IsNullOrWhiteSpace(ElementType)) return false;
        if (ElementIds == null || ElementIds.Count == 0) return false;
        var validTypes = new[] { "vertex", "halfedge", "face", "edge" };
        return validTypes.Contains(ElementType.ToLowerInvariant());
    }
    public IEnumerable<StyleApplyRequest> SplitByAttribute(Func<string, string> attributeResolver)
    {
        if (attributeResolver == null) throw new ArgumentNullException(nameof(attributeResolver));
        // 旧实现：先 Sort 再顺序扫描分组；改用 GroupBy 后语义更直接，且避免修改原列表
        // ElementIds.Sort();
        return ElementIds
            .GroupBy(id => attributeResolver(id) ?? "normal")
            .Select(g => new StyleApplyRequest
            {
```

```
            TargetLayer = TargetLayer,
            ElementType = ElementType,
            ElementIds = g.ToList(),
            StrokeColor = ResolveColor(g.Key, StrokeColor),
            FillColor = ResolveColor(g.Key, FillColor),
            StrokeWidth = g.Key == "nonmanifold" ? StrokeWidth * 2.0 : StrokeWidth,
            ZIndex = g.Key == "nonmanifold" ? 20 : g.Key == "boundary" ? 10 : ZIndex,
            MarkerShape = MarkerShape
        });
    }
    private string ResolveColor(string attribute, string fallback)
    {
        return attribute?.ToLowerInvariant() switch
        {
            "boundary" => "#FF8C00",
            "nonmanifold" => "#DC143C",
            "manifold" => "#808080",
            "selected" => "#4169E1",
            "error" => "#8B0000",
            _ => fallback ?? "#000000"
        };
    }
    public Dictionary<string, object> ToStylePayload()
    {
        return new Dictionary<string, object>
        {
            ["layer"] = TargetLayer,
            ["type"] = ElementType,
            ["ids"] = ElementIds,
            ["stroke"] = StrokeColor,
            ["fill"] = FillColor,
            ["width"] = StrokeWidth,
            ["zIndex"] = ZIndex,
            ["marker"] = MarkerShape
        };
    }
    public static IEnumerable<StyleApplyRequest> FromValidationWarnings(string layer, IEnumerable<string> warnings)
    {
        if (string.IsNullOrWhiteSpace(layer) || warnings == null) yield break;
        var boundaryIds = new List<string>();
        var nonManifoldIds = new List<string>();
        var pattern = new Regex(@"\b(HE\d+|V\d+|F\d+)\b", RegexOptions.Compiled);
        foreach (var msg in warnings)
        {
            if (string.IsNullOrWhiteSpace(msg)) continue;
            var matches = pattern.Matches(msg).Cast<Match>().Select(m => m.Value).Distinct().ToList();
            if (msg.Contains("边界")) boundaryIds.AddRange(matches);
            else if (msg.Contains("非流形")) nonManifoldIds.AddRange(matches);
        }
        if (boundaryIds.Count > 0)
```

```
            yield return new StyleApplyRequest
            {
                TargetLayer = layer,
                ElementType = "halfedge",
                ElementIds = boundaryIds.Distinct().ToList(),
                StrokeColor = "#FF8C00",
                StrokeWidth = 2.5,
                ZIndex = 10
            };
        if (nonManifoldIds.Count > 0)
            yield return new StyleApplyRequest
            {
                TargetLayer = layer,
                ElementType = "halfedge",
                ElementIds = nonManifoldIds.Distinct().ToList(),
                StrokeColor = "#DC143C",
                StrokeWidth = 3.0,
                ZIndex = 20
            };
    }
  }
}
using System;
using System.Collections.Generic;
using System.Linq;
namespace Origami.CreaseTo3D
{
  public class EdgeSplitPreview
  {
    private readonly SnapEngine _snapEngine;
    private readonly VertexKdTree _kdTree;
    public EdgeSplitPreview()
    {
      _snapEngine = new SnapEngine();
      _kdTree = new VertexKdTree();
    }
    public EdgeSplitPreview(SnapEngine engine, VertexKdTree kdTree)
    {
      _snapEngine = engine ?? new SnapEngine();
      _kdTree = kdTree ?? new VertexKdTree();
    }
    /// <summary>
    /// 对折痕边进行 n 等分，并返回吸附后的候选点集合。
    /// </summary>
    public List<SnapCandidate> PreviewEqualSplits(CreaseEdge edge, int divCount, SnapCfgDialog cfg)
    {
      if (edge == null || cfg == null) throw new ArgumentNullException("edge/cfg");
      if (!cfg.Validate()) throw new InvalidOperationException("吸附配置未通过校验");
      if (divCount < 2) throw new ArgumentOutOfRangeException(nameof(divCount), "等分份数至少为 2");
      var result = new List<SnapCandidate>(divCount - 1);
```

```
        Vector3d dir = edge.End.Position - edge.Start.Position;
        double step = 1.0 / divCount;
        for (int i = 1; i < divCount; i++)
        {
            Vector3d raw = edge.Start.Position + dir * (step * i);
            SnapCandidate candidate = _snapEngine.Snap(raw, cfg);
            result.Add(candidate);
        }
        return result;
    }
    /// <summary>
    /// 面片生成前校验闭合回路的几何约束。
    /// </summary>
    public ConstraintResult ValidateFoldGeometry(IReadOnlyList<Vertex> faceLoop, SnapCfgDialog cfg)
    {
        if (faceLoop == null || faceLoop.Count < 3)
            return new ConstraintResult { Satisfied = false, ResidualError = double.MaxValue };
        double totalLen = 0.0;
        double max_error = 0.0;
        for (int i = 0; i < faceLoop.Count; i++)
        {
            Vertex a = faceLoop[i];
            Vertex b = faceLoop[(i + 1) % faceLoop.Count];
            Vertex c = faceLoop[(i + 2) % faceLoop.Count];
            totalLen += SegmentLength(a, b);
            double angle = ComputeInnerAngle(a, b, c);
            if (angle < 0.001)
                max_error = Math.Max(max_error, Math.PI - angle);
        }
        bool ok = totalLen > cfg.MinSegmentLength && max_error < cfg.SnapTolerance * 2.0;
        return new ConstraintResult
        {
            Satisfied = ok,
            ResidualError = max_error
        };
    }
    /// <summary>
    /// 半边翻转的几何可行性：翻转后的新边长必须满足最小分段长度。
    /// </summary>
    public bool CanFlipHalfedge(HalfEdge he, SnapCfgDialog cfg)
    {
        if (he == null || cfg == null || he.IsBoundary) return false;
        if (he.Twin == null || he.Face == null || he.Twin.Face == null) return false;
        Vertex vA = he.Next.Origin;
        Vertex vB = he.Twin.Next.Origin;
        if (vA == null || vB == null) return false;
        double newLen = SegmentLength(vA, vB);
        return newLen >= cfg.MinSegmentLength;
    }
    public void RebuildVertexIndex(List<Vertex> vertices)
```

V1.0

```
    {
        _kdTree.Rebuild(vertices);
        //_kdTree.IncrementalAdd(vertices); // 旧实现：2023-11 前增量构建，边界顶点去重容易出错
    }
    private static double SegmentLength(Vertex a, Vertex b)
    {
        Vector3d d = a.Position - b.Position;
        return Math.Sqrt(d.X * d.X + d.Y * d.Y + d.Z * d.Z);
    }
    private static double ComputeInnerAngle(Vertex a, Vertex b, Vertex c)
    {
        Vector3d ba = a.Position - b.Position;
        Vector3d bc = c.Position - b.Position;
        double dot = ba.X * bc.X + ba.Y * bc.Y + ba.Z * bc.Z;
        double lenA = Math.Sqrt(ba.X * ba.X + ba.Y * ba.Y + ba.Z * ba.Z);
        double lenB = Math.Sqrt(bc.X * bc.X + bc.Y * bc.Y + bc.Z * bc.Z);
        if (lenA < 1e-12 || lenB < 1e-12) return 0.0;
        double cos = Math.Max(-1.0, Math.Min(1.0, dot / (lenA * lenB)));
        return Math.Acos(cos);
    }
}
public class SnapCfgDialog
{
    public bool EnableSnap { get; set; } = true;
    public double SnapTolerance { get; set; } = 0.05;
    public bool PreferBoundaryVertex { get; set; } = true;
    public int MaxCandidates { get; set; } = 3;
    public double MinSegmentLength { get; set; } = 0.01;
    public void LoadDefault()
    {
        EnableSnap = true;
        SnapTolerance = 0.05;
        PreferBoundaryVertex = true;
        MaxCandidates = 3;
        MinSegmentLength = 0.01;
    }
    public bool Validate()
    {
        return SnapTolerance > 0
            && MaxCandidates > 0
            && MinSegmentLength >= 0;
    }
    public void ApplyTo(SnapEngine engine)
    {
        if (engine == null) throw new ArgumentNullException(nameof(engine));
        engine.Configure(this);
    }
    /// <summary>
    /// 由容差推导面片曲率：容差越小越接近刚性纸面。
    /// </summary>
```

```
        public double EstimateCurvature()
        {
            if (!EnableSnap) return 0.0;
            double curvature = SnapTolerance / (SnapTolerance + 0.1);
            return Math.Min(curvature, 1.0);
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Numerics;
namespace Origami.CreaseTo3D
{
    public class ValidationRuleCfg
    {
        public bool AllowBoundaryEdges { get; set; } = true;
        public int MaxAllowedNonManifoldEdges { get; set; } = 0;
        public double MinFaceArea { get; set; } = 1e-6;
        public bool EnableVertexLinkCheck { get; set; } = true;
        public static ValidationRuleCfg Default()
        {
            return new ValidationRuleCfg
            {
                AllowBoundaryEdges = true,
                MaxAllowedNonManifoldEdges = 0,
                MinFaceArea = 1e-6,
                EnableVertexLinkCheck = true
            };
        }
        public void MergeFromDialogInput(bool allowBoundary, double minFaceArea, bool checkVertexLink)
        {
            AllowBoundaryEdges = allowBoundary;
            MinFaceArea = minFaceArea > 0 ? minFaceArea : 1e-6;
            EnableVertexLinkCheck = checkVertexLink;
        }
    }
    public class ManifoldValidator
    {
        private readonly ValidationRuleCfg _cfg;
        private readonly HalfEdgeBuilder _builder;
        public ManifoldValidator(ValidationRuleCfg cfg)
        {
            _cfg = cfg ?? ValidationRuleCfg.Default();
            _builder = new HalfEdgeBuilder();
        }
        public ValidationRuleCfg CurrentCfg => _cfg;
        public TopoReport Validate(IReadOnlyList<CreaseSegment> segments)
        {
            var report = new TopoReport();
```

```
        if (segments == null || segments.Count < 3)
        {
            report.AddError("输入折痕线段数量不足，无法构成最小面片");
            report.SetManifold(false);
            return report;
        }
        var cleaned = DeduplicateSegments(segments);
        report.InputSegmentCount = cleaned.Count;
        try
        {
            var mesh = _builder.Build(cleaned);
            report.FaceCount = mesh.Faces.Count;
            report.HalfEdgeCount = mesh.HalfEdges.Count;
            bool edgeOk = CheckEdgeManifold(mesh, report);
            bool vertexOk = _cfg.EnableVertexLinkCheck && CheckVertexLink(mesh, report);
            bool faceOk = FilterDegenerateFaces(mesh.Faces, report);
            report.SetManifold(edgeOk && vertexOk && faceOk);
        }
        catch (Exception ex)
        {
            report.AddError($"拓扑重建异常：{ex.Message}");
            report.SetManifold(false);
        }
        return report;
    }
    private List<CreaseSegment> DeduplicateSegments(IReadOnlyList<CreaseSegment> segments)
    {
        var seen = new HashSet<string>();
        var result = new List<CreaseSegment>(segments.Count);
        foreach (var seg in segments)
        {
            if (seg == null) continue;
            var key = NormalizeEdgeKey(seg.Start, seg.End);
            if (seen.Add(key))
            {
                result.Add(seg);
            }
        }
        return result;
    }
    private string NormalizeEdgeKey(Vector3 a, Vector3 b)
    {
        var k1 = $"{a.X:F6},{a.Y:F6},{a.Z:F6}";
        var k2 = $"{b.X:F6},{b.Y:F6},{b.Z:F6}";
        return string.Compare(k1, k2, StringComparison.Ordinal) < 0
            ? $"{k1}|{k2}"
            : $"{k2}|{k1}";
    }
    private bool CheckEdgeManifold(HalfEdgeMesh mesh, TopoReport report)
    {
```

```
        var incidence = new Dictionary<(int v0, int v1), int>();
        int violationCount = 0;
        foreach (var face in mesh.Faces)
        {
            foreach (var he in face.HalfEdges)
            {
                int a = he.OriginIndex;
                int b = he.Next.OriginIndex;
                var key = a < b ? (a, b) : (b, a);
                incidence[key] = incidence.GetValueOrDefault(key, 0) + 1;
            }
        }
        foreach (var pair in incidence)
        {
            int count = pair.Value;
            if (count > 2)
            {
                report.AddNonManifoldEdge(pair.Key, count);
                violationCount++;
            }
            else if (count == 1 && !_cfg.AllowBoundaryEdges)
            {
                report.AddBoundaryViolation(pair.Key);
                violationCount++;
            }
        }
        // var old_directed_check = incidence.Count; // 旧实现按有向边统计，会把闭合边误判为两条
        return violationCount <= _cfg.MaxAllowedNonManifoldEdges;
    }
    private bool CheckVertexLink(HalfEdgeMesh mesh, TopoReport report)
    {
        bool allOk = true;
        var outgoing = mesh.HalfEdges.GroupBy(h => h.OriginIndex).ToList();
        foreach (var group in outgoing)
        {
            int vertex = group.Key;
            var starts = group.ToList();
            int boundaryCount = starts.Count(h => h.Twin == null);
            if (boundaryCount > 2)
            {
                report.AddVertexLinkError(vertex, "顶点处边界边超过两条，流形不成立");
                allOk = false;
                continue;
            }
            if (!_cfg.AllowBoundaryEdges && boundaryCount > 0)
            {
                report.AddVertexLinkError(vertex, "存在边界边但当前规则禁止边界");
                allOk = false;
            }
        }
```

```
            return allOk;
        }
        private bool FilterDegenerateFaces(IReadOnlyList<HalfEdgeFace> faces, TopoReport report)
        {
            bool ok = true;
            foreach (var face in faces)
            {
                double area = ComputeFaceArea(face);
                if (area < _cfg.MinFaceArea)
                {
                    report.AddDegenerateFace(face.Id, area);
                    ok = false;
                }
            }
            return ok;
        }
        private double ComputeFaceArea(HalfEdgeFace face)
        {
            var verts = face.Vertices.ToList();
            if (verts.Count < 3) return 0.0;
            double area = 0.0;
            var origin = verts[0];
            for (int i = 1; i < verts.Count - 1; i++)
            {
                var a = verts[i] - origin;
                var b = verts[i + 1] - origin;
                area += 0.5 * Vector3.Cross(a, b).Length();
            }
            return area;
        }
    }
}
namespace Origami.CreaseTo3D
{
    using System;
    using System.Collections.Generic;
    using System.Linq;
    public class SortCfgRequest
    {
        public string ModelCode { get; set; }
        public string[] SelectedFaceIds { get; set; }
        public Dictionary<string, double> CreaseFoldAngles { get; set; }
        public Dictionary<string, bool> MountainValleyFlags { get; set; }
        public Dictionary<string, (double x, double y, double z)> FoldAxisDirections { get; set; }
        public bool PreserveLayerStacking { get; set; }
        public double AngularTolerance { get; set; }
        public string Strategy { get; set; }
    }
    public class FaceSequencePlanner
    {
```

V1.0

```
    private readonly DependencyGraph _depGraph;
    private readonly HalfEdgeOrderCalc _orderCalc;
    private readonly TopoOptimizer _optimizer;
    public FaceSequencePlanner()
    {
        _depGraph = new DependencyGraph();
        _orderCalc = new HalfEdgeOrderCalc();
        _optimizer = new TopoOptimizer();
    }
    public List<(string faceId, int sequence, double stress)> PlanFaceSequence(SortCfgRequest req)
    {
        Validate(req);
        var rawOrder = _orderCalc.Compute(req.SelectedFaceIds, _depGraph).ToList();
        var layers = BuildLayerBuckets(req);
        var scored = rawOrder
            .Select(f => (faceId: f, stress: EstimateFoldStress(f, req)))
            .ToList();
        // var seq = rawOrder.Select((id, idx) => (id, idx + 1, 0.0)).ToList();
        // 旧实现直接返回拓扑序，未考虑折叠应力与层叠约束，2024-03 后废弃
        var merged = MergeLayerAndStress(layers, scored, req.PreserveLayerStacking);
        var optimized = _optimizer.Optimize(merged, req.CreaseFoldAngles);
        int seq = 1;
        return optimized.Select(f => (f.faceId, seq++, f.stress)).ToList();
    }
    public SortCfgRequest BindFromViewport(
        string modelCode,
        string faceId,
        double foldAngle,
        bool isValley,
        (double x, double y, double z) axis)
    {
        if (string.IsNullOrEmpty(faceId))
            throw new ArgumentException("faceId 不能为空", nameof(faceId));
        return new SortCfgRequest
        {
            ModelCode = modelCode,
            SelectedFaceIds = new[] { faceId },
            CreaseFoldAngles = new Dictionary<string, double> { [faceId] = foldAngle },
            MountainValleyFlags = new Dictionary<string, bool> { [faceId] = isValley },
            FoldAxisDirections = new Dictionary<string, (double, double, double)> { [faceId] = axis },
            PreserveLayerStacking = true,
            AngularTolerance = 0.01,
            Strategy = "layer_stress"
        };
    }
    private void Validate(SortCfgRequest req)
    {
        if (req == null)
            throw new ArgumentNullException(nameof(req));
        if (req.SelectedFaceIds == null || req.SelectedFaceIds.Length == 0)
```

```
            throw new ArgumentException("未指定待排序面片");
        req.CreaseFoldAngles ??= new Dictionary<string, double>();
        req.MountainValleyFlags ??= new Dictionary<string, bool>();
        req.FoldAxisDirections ??= new Dictionary<string, (double, double, double)>();
        foreach (var id in req.SelectedFaceIds)
        {
            if (req.CreaseFoldAngles.TryGetValue(id, out var a) && (a < -180 || a > 180))
                throw new ArgumentOutOfRangeException($"面片 {id} 折叠角度越界");
        }
        if (req.AngularTolerance <= 0)
            req.AngularTolerance = 1e-3;
    }
    private List<List<string>> BuildLayerBuckets(SortCfgRequest req)
    {
        var baseNormal = req.FoldAxisDirections.Values.Any()
            ? req.FoldAxisDirections.Values.Aggregate((a, b) => (a.x + b.x, a.y + b.y, a.z + b.z))
            : (0.0, 0.0, 1.0);
        var buckets = req.SelectedFaceIds
            .GroupBy(f => req.FoldAxisDirections.TryGetValue(f, out var d)
                ? Math.Round(Dot(d, baseNormal) / Length(d), 2)
                : 0.0)
            .OrderBy(g => g.Key)
            .Select(g => g.ToList())
            .ToList();
        return buckets;
    }
    private List<(string faceId, double stress)> MergeLayerAndStress(
        List<List<string>> layers,
        List<(string faceId, double stress)> scored,
        bool preserveLayer)
    {
        var lookup = scored.ToDictionary(x => x.faceId, x => x.stress);
        var merged = new List<(string faceId, double stress)>();
        if (preserveLayer)
        {
            foreach (var layer in layers)
            {
                merged.AddRange(layer
                    .Where(lookup.ContainsKey)
                    .OrderBy(f => lookup[f])
                    .Select(f => (f, lookup[f])));
            }
        }
        else
        {
            merged.AddRange(scored.OrderBy(x => x.stress));
        }
        return merged;
    }
    private double EstimateFoldStress(string faceId, SortCfgRequest req)
```

V1.0

```
        {
            double angle = req.CreaseFoldAngles.TryGetValue(faceId, out var a) ? Math.Abs(a) : 0.0;
            bool isValley = req.MountainValleyFlags.TryGetValue(faceId, out var v) && v;
            // 山折在锐角时更易产生回弹误差，排序后置以减小累积
            double stress = isValley ? angle * 0.8 : angle * 1.2;
            return Math.Round(stress, 4);
        }
        private static double Dot((double x, double y, double z) a, (double x, double y, double z) b)
        {
            return a.x * b.x + a.y * b.y + a.z * b.z;
        }
        private static double Length((double x, double y, double z) v)
        {
            return Math.Sqrt(v.x * v.x + v.y * v.y + v.z * v.z);
        }
        private bool PreCheckCreaseConsistency(SortCfgRequest req)
        {
            return req.SelectedFaceIds.All(f => req.CreaseFoldAngles.ContainsKey(f));
        }
    }
}
```